

Lab 4: ROM Based Game Access Control on FPGA Board

Abdur Aziz

Course Number : ECE 5440

Introduction:

The Lab 4 project builds upon the Access Controller system developed in Lab 3, expanding it into a modular, single-player game implemented on an FPGA development board. This version introduces two technical advancements: (1) the separation of the timer control logic from the password authentication function, and (2) the use of a ROM to securely store the predefined access password. These architectural improvements enhance both the design scalability and hardware efficiency, while keeping the user interface and game flow consistent with Lab 3.

To begin gameplay, the user must first authenticate by entering a four-digit passcode, “2062,” using push-button inputs connected to the FPGA. These digits are debounced using a Button Shaper module, captured using Load Registers, and validated through a comparison process where each input is checked against a corresponding ROM-stored digit. Only upon successful authentication does the system transition to gameplay mode.

Once logged in, the system initializes a random number using a Linear Feedback Shift Register (LFSR)-based RNG module, which simulates a bot's movement. The generated number is shown on a seven-segment display. The player then selects their own input via switches, aiming to compute a result that aligns with or challenges the bot's value. An Adder module processes both values, and the sum is displayed on a dedicated seven-segment display.

Timing regulation is managed by a newly modularized Timer Controller, which communicates with an LFSR-based timer to enforce a strict time limit for each round. Status LEDs provide real-time feedback on login status, game state, and the outcome of each interaction, while Seven-Segment Decoder modules offer clear visualization of inputs and results throughout the game.

This project showcases advanced embedded system design concepts, including secure access control, random number generation, real-time user interaction, and dynamic game logic implementation. By integrating these modules through Verilog HDL, Lab 4 demonstrates how FPGA platforms can be used to create engaging, hardware-based interactive games that combine authentication mechanisms using ROM systems with digital gameplay experiences.

This game was designed using Verilog language. Figure 1.1 below shows the FPGA board that the mental binary game was designed and displayed on.

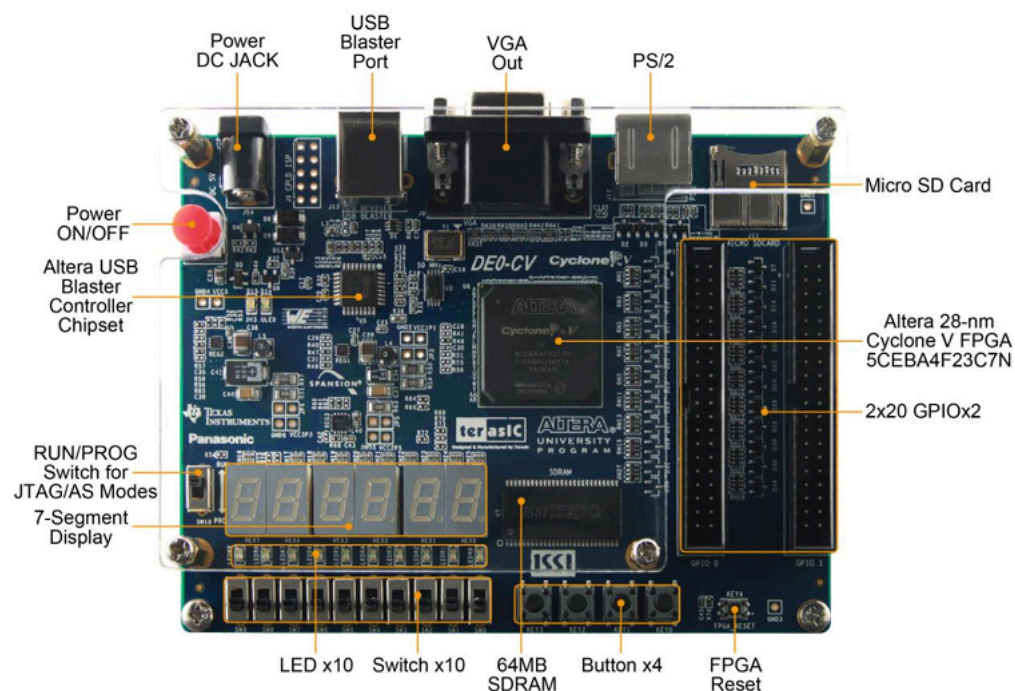


Figure 1.1: FPGA Board Game

It's important to note that while the overall architecture from Lab 2 was preserved, the Access Controller functionality was modularized into two distinct components: the **Authentication Module**, responsible for verifying the user's passcode using ROM-based storage, and the Game ControllerModule, which manages game state and timer control. Only two outputs — Timer_reconfig and Timer_enable — are sent from the Game Controller Module to the OneSecondTimer module, which is itself composed of three cascaded timing units: a One_MSLFSR (LFSR-based 1 ms timer), a timer_10ms, and a CountTo100 module. The output tick from the OneSecondTimer feeds into the TwoDigitTimer, which is constructed using two cascaded DigitTimer modules. This modular design enables clean separation between authentication, game control, and timekeeping functions, improving system clarity and reusability. **Figure 1.2 below shows the high level system architecture for Lab 3.**

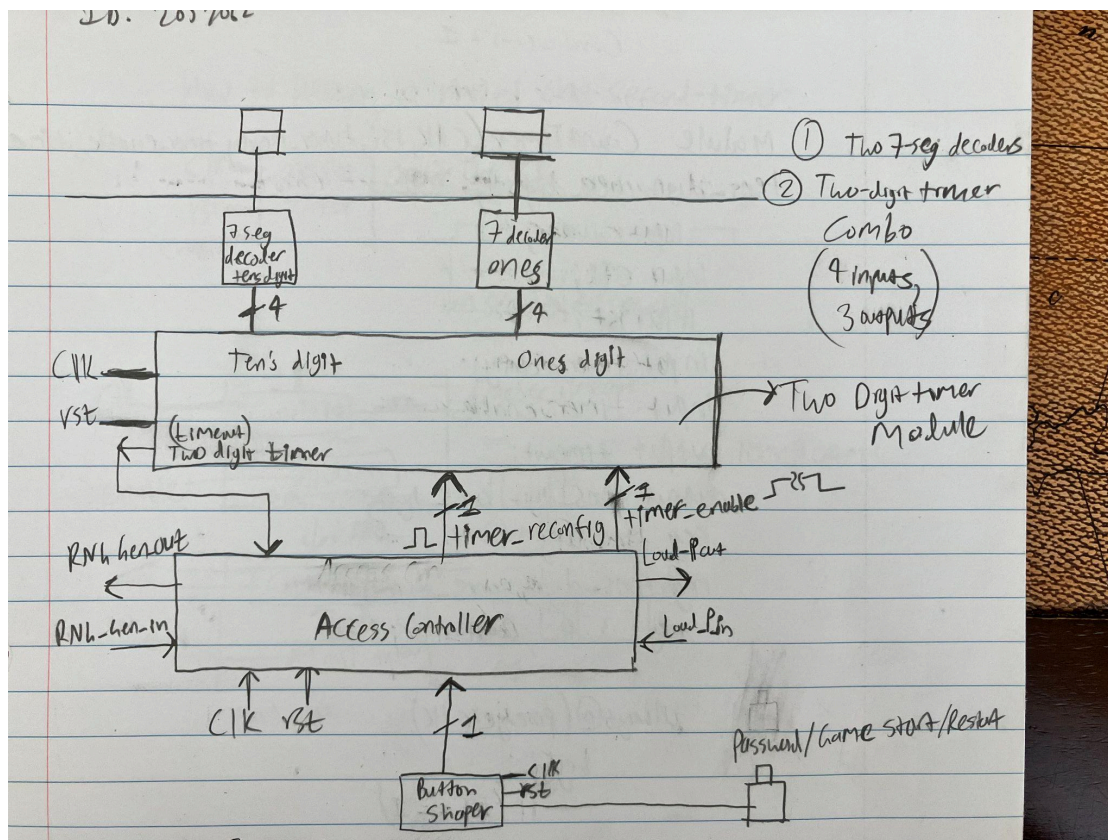


Figure 1.2: Lab 3 High Level System Architecture

Figure 1.3 below shows the high level system architecture for Lab 4's access controller split into the Game Controller module and the Authenticator module.

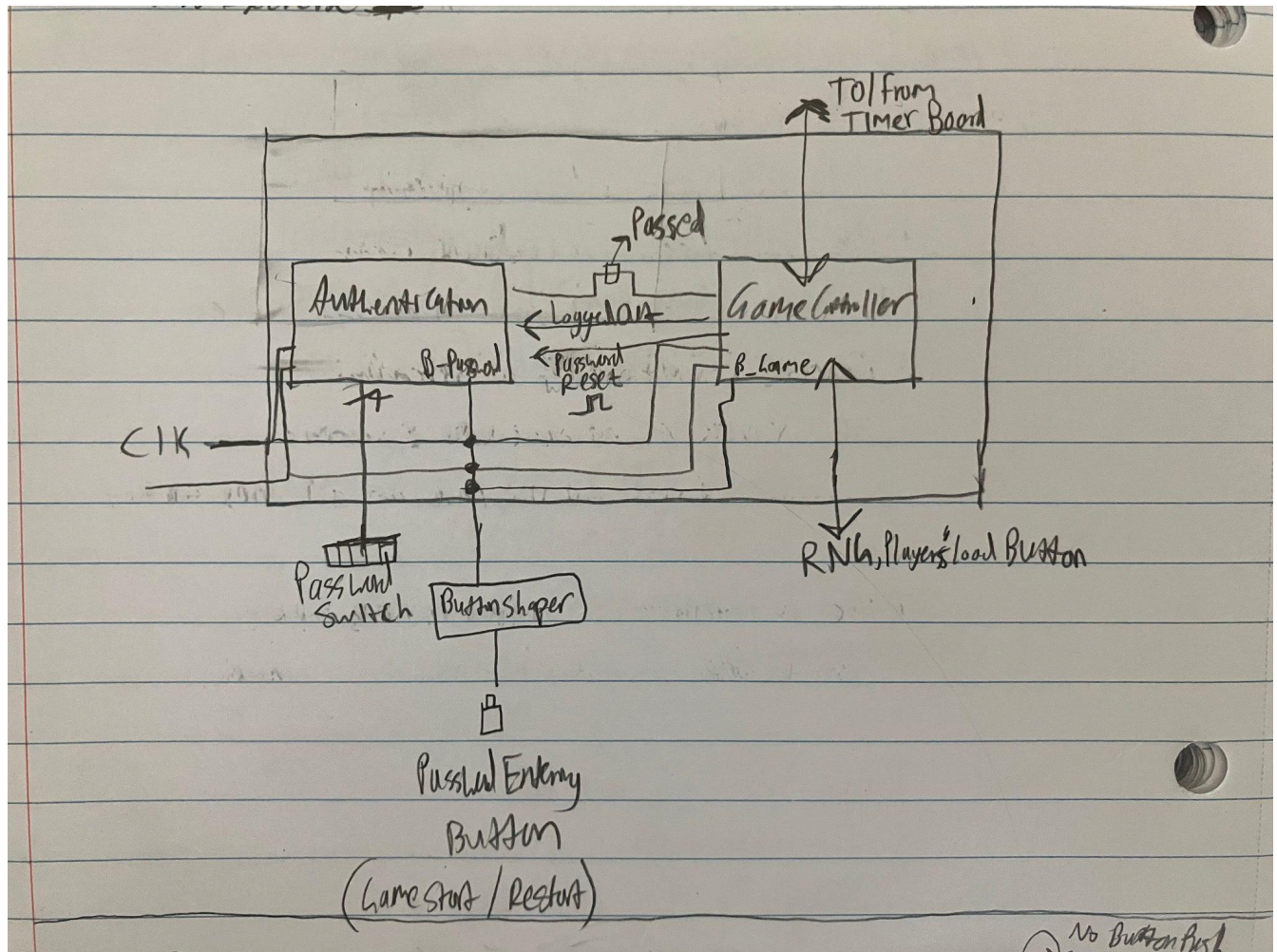


Figure 1.3: Lab 4's High Level System Architecture for the Split Access Controller

It's important to break down the high-level system architecture and explain how each module contributes to the overall functionality. In Lab 4, the original Access Controller from Lab 2 has been modularized into two key components: the Authentication Module and the Game Controller Module. The authentication process

builds upon the secure password mechanism introduced in Lab 2, but is now optimized for a single-player environment. In this setup, the player must enter a predefined four-digit passcode, “2062,” using physical input switches on the FPGA board. Each digit is passed through a Button Shaper module to eliminate mechanical switch noise and stabilize the input signal. These sanitized values are captured sequentially by the Load Register module.

Once all four digits are entered, the Authentication Module compares the player's input against a preset password stored in a ROM. If the passcode matches, a LoggedIn signal is asserted and passed to the Game Controller Module, which then transitions the system into gameplay by enabling game control modules such as the timer and RNG. If the input is incorrect, the system remains in a locked state, prompting the player to retry authentication.

Unlike the original design in Lab 2, which supported two players with distinct roles and authentication paths, this modified version focuses solely on a single player's authentication flow, simplifying the logic while preserving secure access control. This ensures that the game remains protected against unintended access and that gameplay can only commence following successful user verification. **It's important to note that Lab 4 is a single player game, and Player 2 is replaced with the Random Number Generator, which will be discussed later in this lab. As per Lab 4, Player 1 only uses the password input switches and the sum switches.**

Figure 1.4 below shows the original access controller system architecture from Lab 3 for curiosity objectives. For more information about the ROM code and its testbench, please peek into the project file labeled ROM_PSWD.v.

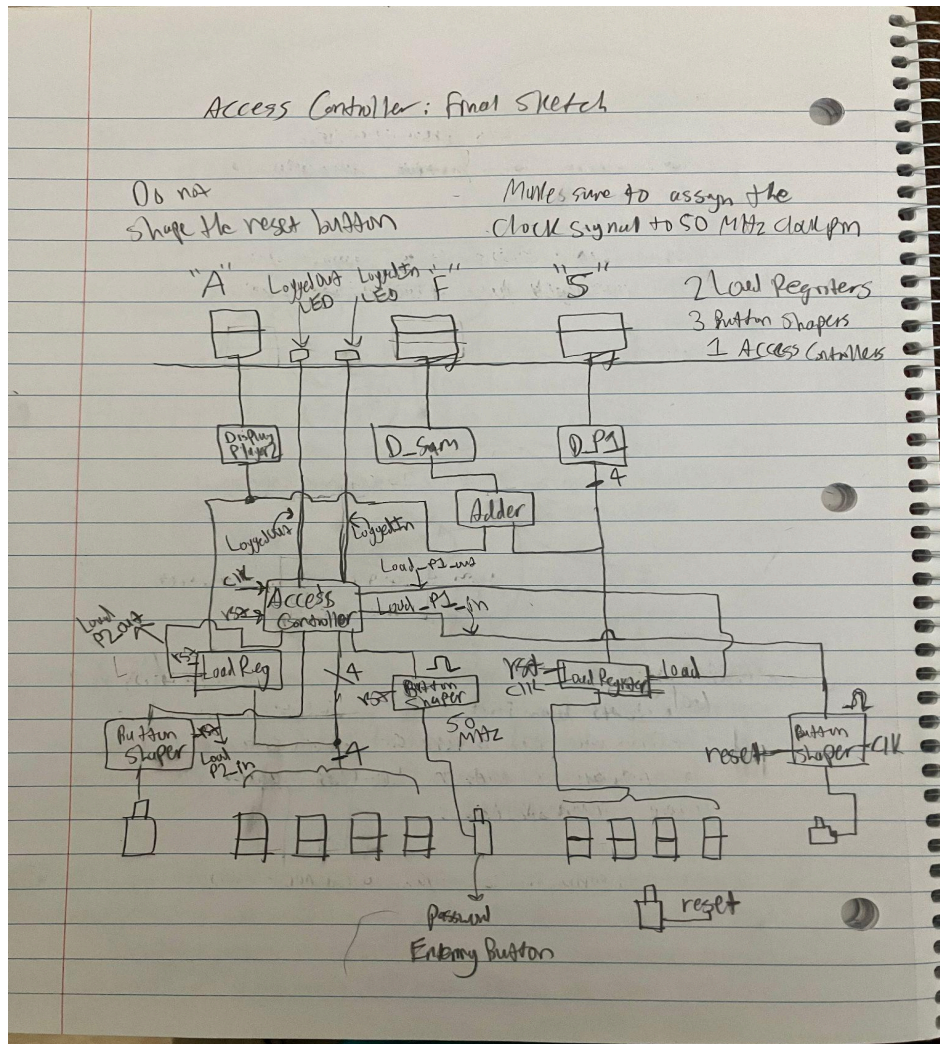


Figure 1.4: Access Controller System Original Architecture before split

The **Digit Timer module** serves as a crucial timing controller within the FPGA design, primarily responsible for generating slower timing pulses derived from the high-frequency system clock. Since the FPGA board operates at a fast clock rate (such as 50 MHz), the Digit Timer reduces this frequency to create usable timing signals for display refreshing and game logic pacing. Internally, the module utilizes a counter register that increments on every positive edge of the system clock, but only when the enable signal is asserted. When the counter reaches a predefined maximum value—determined by the desired time interval—it resets back to zero and emits a

one-clock-cycle tick pulse. This tick acts as a timing pulse that triggers subsequent actions, such as advancing game states or refreshing the digits on the seven-segment displays.

The module accepts three input signals: the system clock (clk), a reset signal (reset), and an enable control (enable). The reset input allows the counter and tick output to be cleared immediately, ensuring reliable startup or recovery from faults. The enable signal provides control over when the timer operates; when enable is low, the counter halts and resets, conserving power and avoiding unnecessary operations during idle or authentication phases of the game. When enabled, the counter accumulates clock cycles until it reaches the terminal count, producing the tick output that can then drive other modules such as the display multiplexer or the game's finite state machine.

In the context of the game, the Digit Timer module plays a vital role in regulating the pace of user interactions and system feedback. It ensures that display updates, player inputs, and game state transitions occur at intervals that are observable and manageable for the player. **Figure 1.5 below shows the high level connection between the access controller and the digit timer module.**

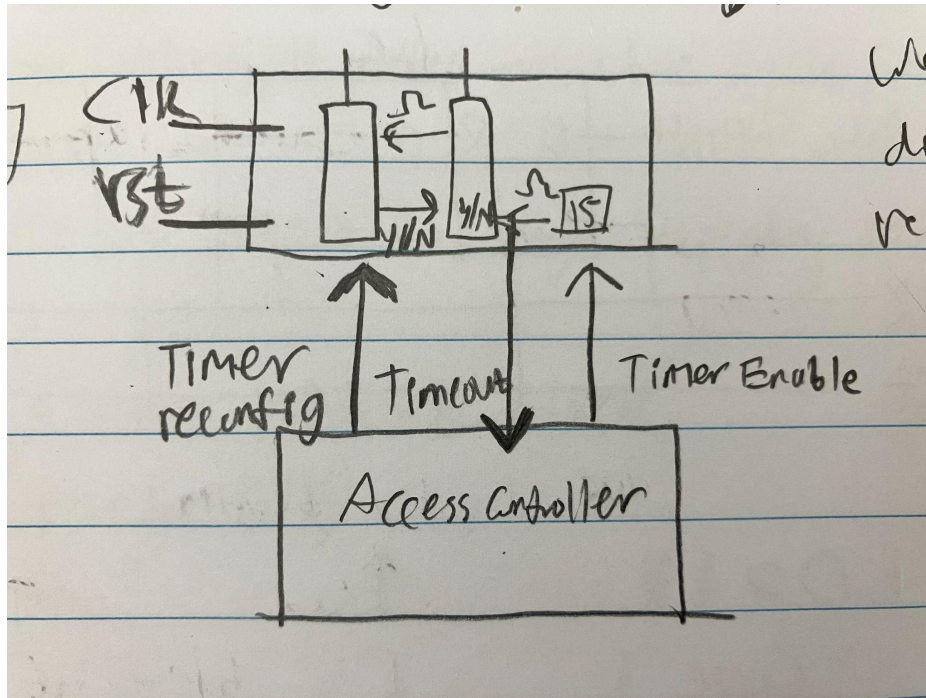


Figure 1.5: Digit Timer module connection with Access Controller module

It's important to note that the Digit Timer module accepts and outputs additional signals as well: BorrowDown, NoBorrowDown, UpBorrow, and NoUpBorrow. These signals play a critical role in determining the counter's behavior and in signaling when counting events, such as underflow or overflow, occur.

The **BorrowDown** signal is activated when the counter is in **down-counting mode** and reaches zero. This condition signifies that the counter has completed its downward count and needs to signal to higher-level timers or state machines that a timing event has occurred. Borrowing, in this context, is similar to the concept of cascading counters in multi-digit systems, where a lower digit borrows from a higher one when it underflows.

In contrast, **NoBorrowDown** indicates that the counter is actively counting down but has **not yet reached zero**. This signal helps maintain the normal decrementing

operation without triggering any carry-over or state changes in connected modules. It ensures that no premature borrow signals are sent while the counter is still active.

The **UpBorrow** signal works similarly but applies to the **up-counting mode** of the counter. When the counter increments to its maximum predefined value, the UpBorrow signal is asserted to indicate that an upward counting threshold has been reached. This condition can be used to cascade into higher-order timers or to generate timebase signals for the game's logic transitions and display updates.

Finally, **NoUpBorrow** denotes that the counter is in the process of counting upward but has **not yet reached the terminal count value**. This signal ensures the counter continues normal operation without issuing any premature overflow signals, preserving the integrity of the timing flow within the system.

Together, these four signals form the essential control mechanism of the Digit Timer. They enable the module to function flexibly in both up-counting and down-counting configurations, facilitate cascading multiple timers for complex timekeeping, and provide precise control over timing events. Figure 1.6 below shows the Digit Timer module.

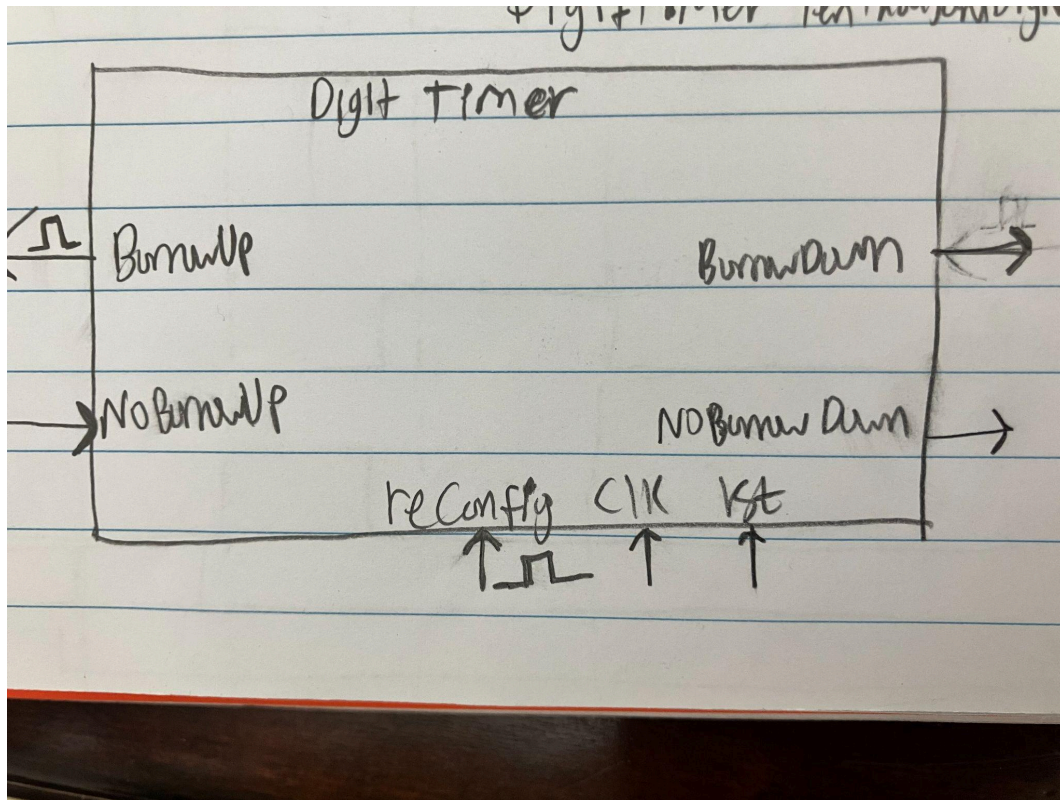


Figure 1.6 : Digit Timer module

Additionally, the One Second Timer module encompasses the Digit Timer module. As mentioned previously at the beginning of this introduction, the **one** millisecond LSFR based module, **timer_10ms** module, and **Count-to-100** millisecond module are included in the One Second Timer module as well.

The **timer_10ms** module, named as countTo10 module in Figure 1.7 below, takes the output of the countTo1 module as its input clock. Rather than counting raw system clock cycles, it counts the number of ticks produced by countTo1. When countTo10's counter reaches its set count (typically 10), it emits its own single-cycle pulse. This design effectively creates a larger timebase, such as a 10-millisecond interval.

The **CountTo100** module receives its input clock from the output of countTo10. It counts the number of pulses from countTo10 and generates a single pulse after accumulating its terminal count, often set to 10 or 100 depending on timing requirements. This cascading structure allows the system to create precise, human-perceivable timing intervals from an initially fast FPGA clock source.

The One Second Timer module includes the instantiations of the 3 modules above. **Figure 1.7 below shows the system architecture for Lab 3's modules. Please refer to Figure 3.2 in the appendix for Lab 4's One millisecond LFSR based code.**

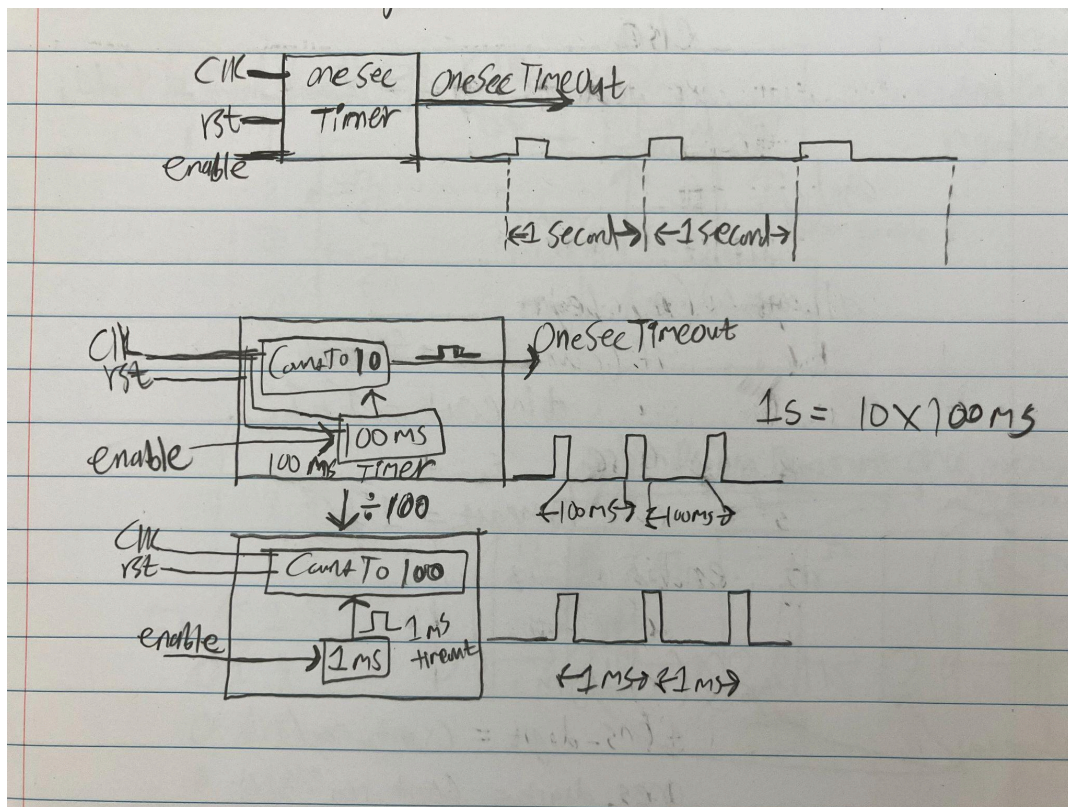


Figure 1.7 : One Second Timer, countTo100, countTo10 modules

The last modules in this design are the Random Number Generator module and the Four Bit Counter Module. The **Random Number Generator (RNG) module** in the game is responsible for producing unpredictable numbers that simulate the bot's move during gameplay. Internally, the RNG operates based on a linear feedback shift register

(LFSR), which continuously cycles a sequence of bits using shift and feedback operations to generate a pseudo-random sequence. Each time the RNG button is pressed by the user, the current state of the register is captured, and the resulting number is displayed on the designated seven-segment display as the bot's random number. This adds an element of unpredictability to the game, ensuring that each round presents a unique challenge for the player while maintaining hardware efficiency within the FPGA design.

The **Four Bit Counter module** is a simple counting mechanism that increments its value with each press of the designated button, counting from zero up to a maximum value of fifteen due to its four-bit capacity. When the counter reaches its maximum value and the button is pressed again, it rolls over back to zero, creating a continuous loop of counts. The module operates synchronously with the system clock and only increments when enabled, allowing controlled counting based on user input or game state transitions.

It's important to note that the random number generator is taken in as an input for the Four Bit Counter Module. The count_out is the output for the Random Number Generator module. Figure 1.8 below depicts the system architecture for the Random Number generator and the Four Bit Counter Module.

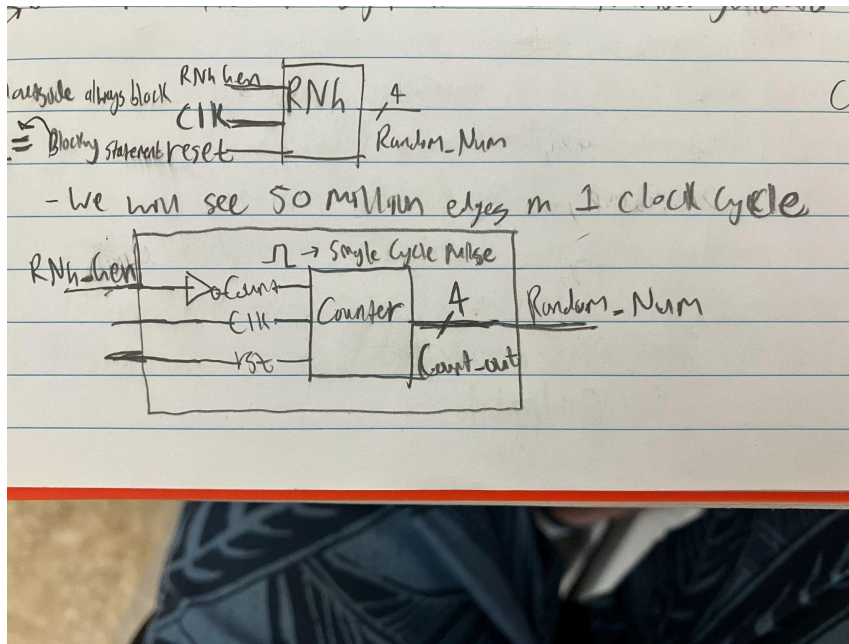


Figure 1.8: Random Number Generator Module and Four Bit Counter Module

Once the correct password is entered and the timer begins to start ticking from 99 to 0, the user can start adding numbers. Figure 1.8 below shows the top module system architecture design for the mental math based binary game. There are two 7 segment decoder modules and 1 4 bit adder decoder module. Player 1's switches are the rightmost 4 bits (S1 - S4), and Player 2's switches are the leftmost switches (S5 - S8). The seven segment decoders connect to each player's respective LED display. **Figure 1.9 below shows the in-depth system architecture for the mental math binary game function for Lab 3 and Lab 4.**

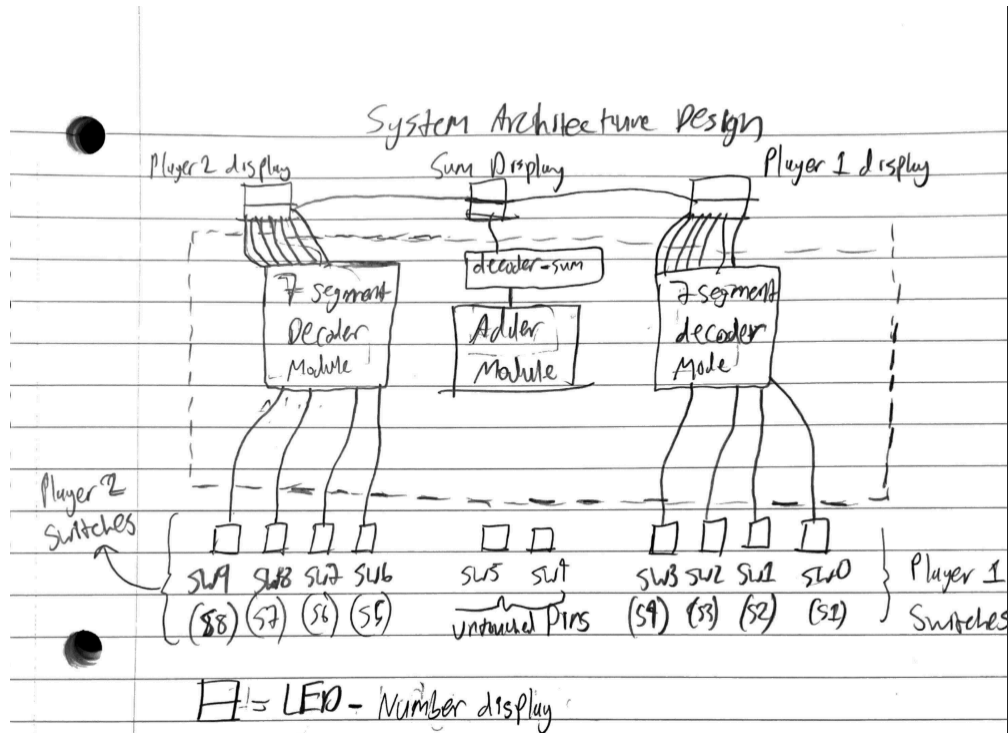


Figure 1.9 : In-depth system design for Mental Math Binary Game Function

Figure 1.10 below shows the Seven Segment Decoder Module. This is representative for each of the two seven segment decoder modules. From the top to the bottom, Decoder In 1 is the least significant bit, and Decoder In 4 is the most significant bit. They serve as input signals going to the breadboard. The same applies for the output decoders, with Decoder Out 7 being the most significant bit. They serve as output signals going to the display. The decoder takes in a 4 bit binary number and outputs a 7 bit binary number to display on clocks, digital circuit boards, etc. **It's important to note that for Lab 4, the Seven Segment Display module displays the ones digit and the tens digit for the Digital Timer and the random number generated on the FPGA board.**

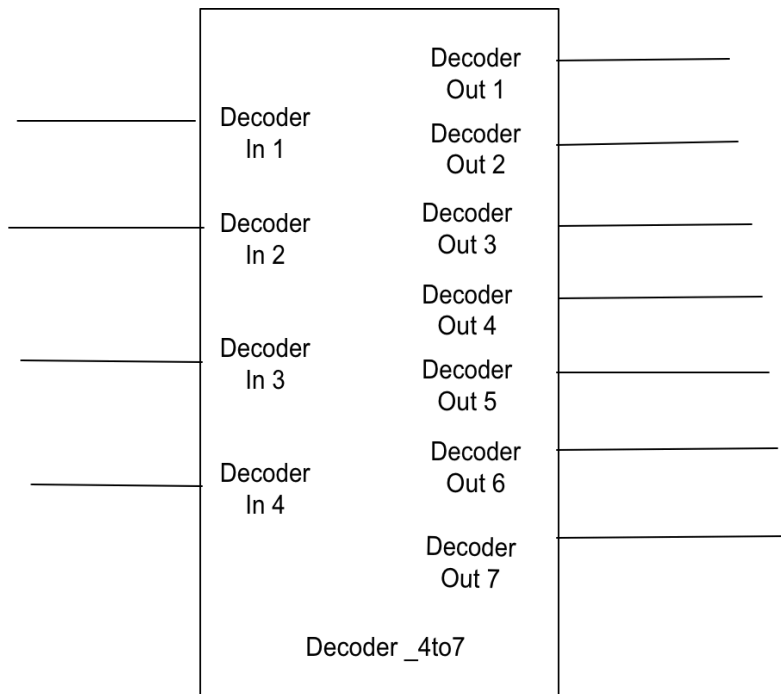


Figure 1.10: Seven Segment Decoder Module.

Figure 1.11 below shows the 4 bit adder decoder module diagram. A_s is a 4 bit binary number, and B_s is a 4 bit binary number. Both serve as input signals. S_s is a 4 bit binary number which represents the sum of A_s and B_s . S_s server as the output signal.

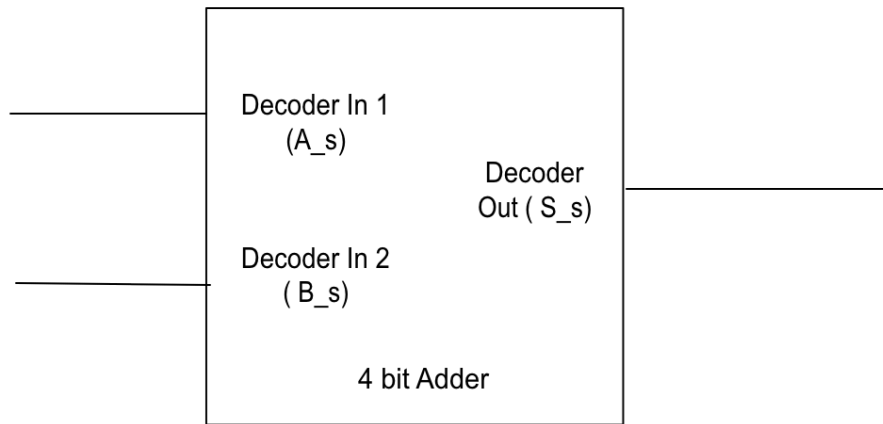


Figure 1.11 : Adder module

Figure 2.1 below shows the diagram of the Button Shaper Module. There are 3 inputs : B_in, clk, and rst. B_in serves as the input when the button is pressed, clk is the function of how the clock works, and rst is when the button goes back to the default state or continues to proceed in the sequence shown by the 3 right block arrows. Finally, B_out is the final state of the button whether the button was pressed or still at its default state.

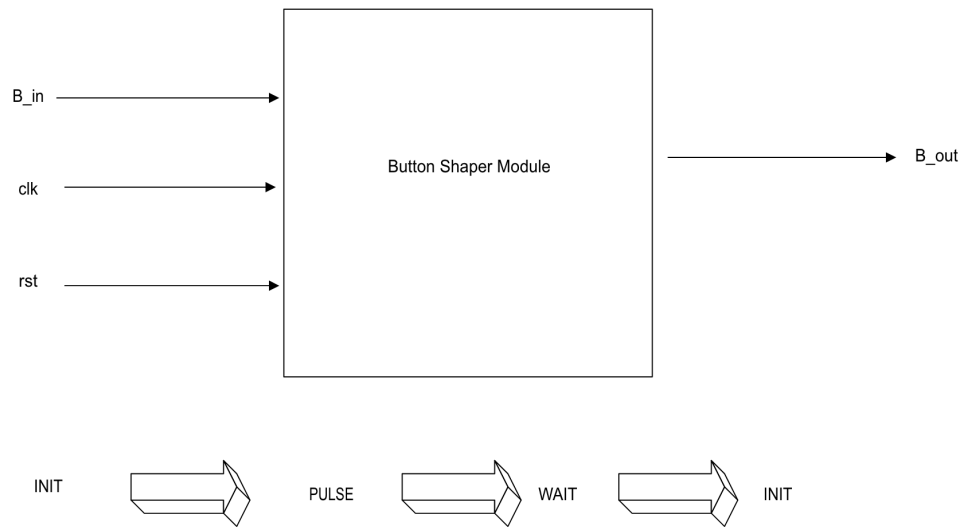


Figure 2.1: Button Shaper Module

The next component is the Load Register, which, in the FPGA board game, functions as a temporary storage unit that captures and holds the user's input data for processing. When a player presses the buttons on the board, the signals are first passed through the ButtonShaper module to eliminate any noise or bouncing effects that naturally occur with mechanical button presses. Once the signals are clean, they are sent to the Load Register. However, the Load Register does not automatically store data; it waits for a specific control signal, known as the "load" signal. When this load signal is activated, the register captures the current state of the inputs at that exact moment and stores them internally. This captured data remains stable and unchanged, even if the user continues to press or release buttons afterward.

The stored values are then made available to other modules in the system like the Authenticator Module, which uses them to verify if the correct input sequence matches the predefined password. Additionally, if the game includes arithmetic operations like tracking scores or attempts, the stored data can also be sent to the AdderDecoder for processing. The Load Register ensures that data is securely held and reliably delivered to other parts of the system, contributing to smooth and accurate game operation. It also responds to system resets by clearing its contents, preparing itself to capture fresh inputs for the next round of gameplay.

Simulation Results: The waveforms all seemed fine. For the TwoDigit waveform, the tens digit changed after the ones digit transitioned from 0 to 9, indicating that the digit timer works. Additionally, the tens digit was able to decrement by 1 successfully from n to 0, where n is a digit from 0 - 9. For instance, 99, 98, 97...91, 90, 89 is what was observed from the waveform when the tens digit made a transition. The ones digit was always changing. As for the remaining modules, they all seemed to be working correctly.

The testbench for Lab 4 validates the authentication and game control logic by simulating a complete single-player interaction on the FPGA. The design is now split into two distinct modules—Authentication Module for secure password entry and Game Controller Module for managing game state and timer control—both of which are derived from the original Access Controller developed in Lab 2. The simulation begins by initializing all input signals and issuing a reset to ensure a predictable starting state. It

then emulates a user entering the correct four-digit passcode “2-0-6-2” by applying each digit sequentially and simulating presses of the PasswordEnterButton. As each digit is entered, the Authentication Module compares it against the stored password values retrieved from ROM, and transitions through internal states accordingly.

FPGA Board Testing:

To set up the FPGA Board, the Type A to B USB was connected from the computer’s USB port to the FPGA Board’s USB Blaster Port. Next, the Power DC Adapter is connected from the Power DC Jack to a nearby wall plug. Figure 2.2 below shows the setup of the FPGA Board.

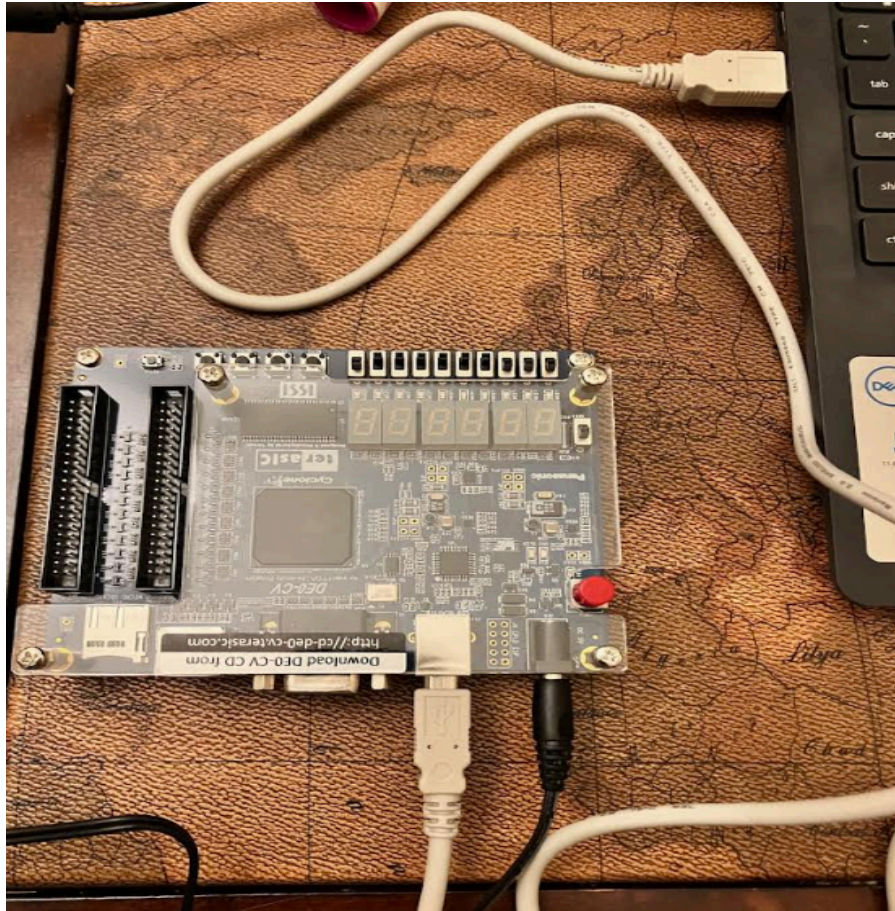


Figure 2.2: FPGA Board Setup

Once the FPGA board is turned on, by default, the Log Out LED is turned on, which is the default state. Additionally, only zeros are showing on the board. Figure 2.25 below depicts this.

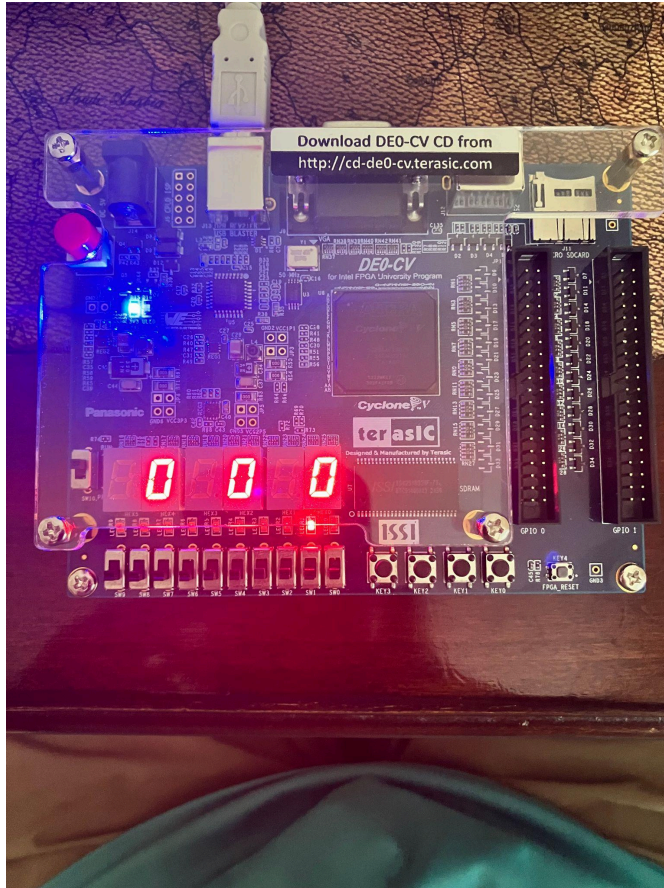


Figure 2.25: FPGA Board Powered On

To begin the game, the user presses the **POWER ON/OFF** button to turn on the FPGA board. Next, the user will enter their correct password, which is **2062**. Looking from right to left, the last right 4 toggle switches - assuming the leftmost bit is the MSB and the rightmost bit is the LSB - are used to enter the password. First, 2 (0010) is inputted using the toggle switches, and the Password entering button is pressed. Next, 0(0000) is inputted using the toggle switches, and the Password entering button is pressed. Next, 6(0110) is inputted using the toggle switches, and the Password entering button is pressed. Finally, 2(0010) is inputted using the toggle switches, and the **“Logged**

In” LED switches on. In a case where the password input is incorrect, the user needs to try the entire process again.

Figure 2.3 below shows the FPGA board in LoggedIn state since the rightmost LED is turned on. Additionally, we can see the number “99” in the timer display.

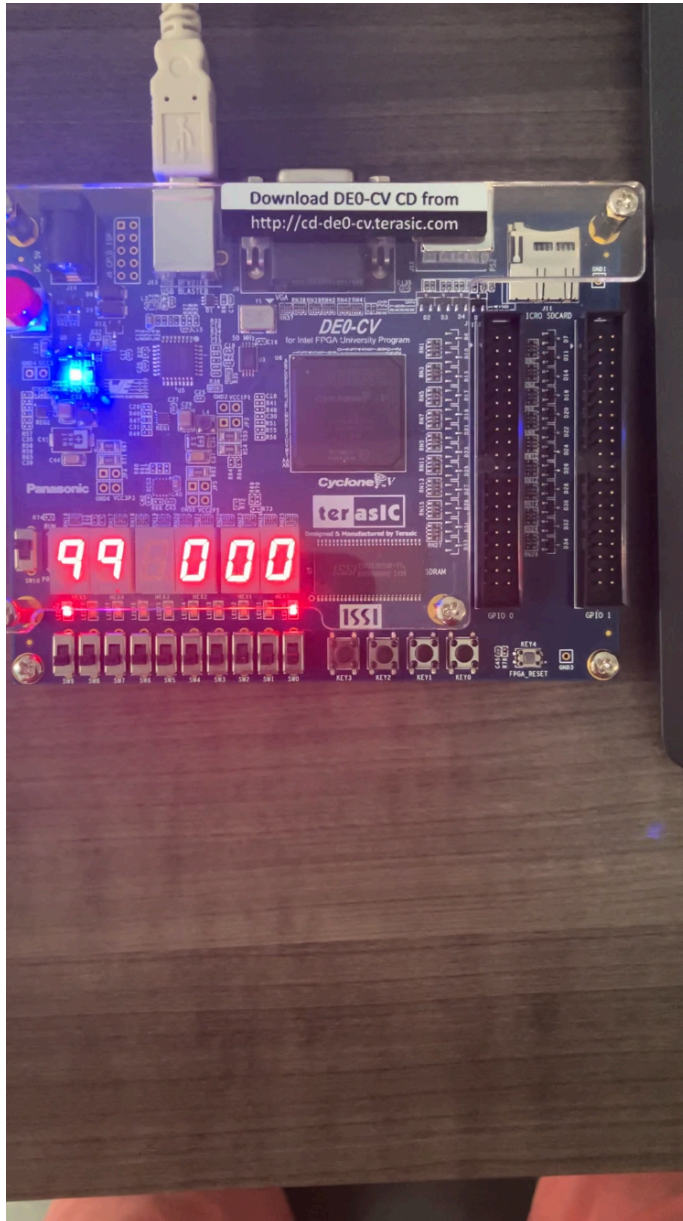


Figure 2.3: FPGA Board Ready-To-Play

Figure 2.4, 2.5, 2.6, 2.7, 2.8, and 2.9 all below depict the Lab 4 module working as a function of time. In other words, these figures prove that the user can add numbers from the random number generator and from the player input switches.

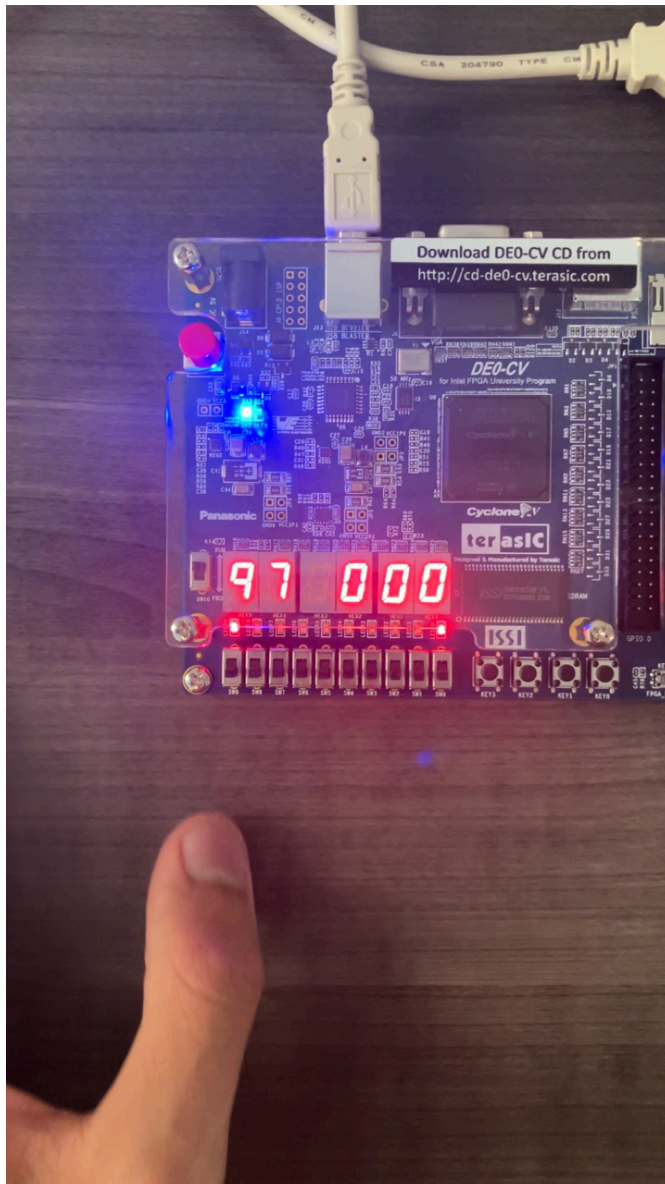


Figure 2.4 : Timer counting down

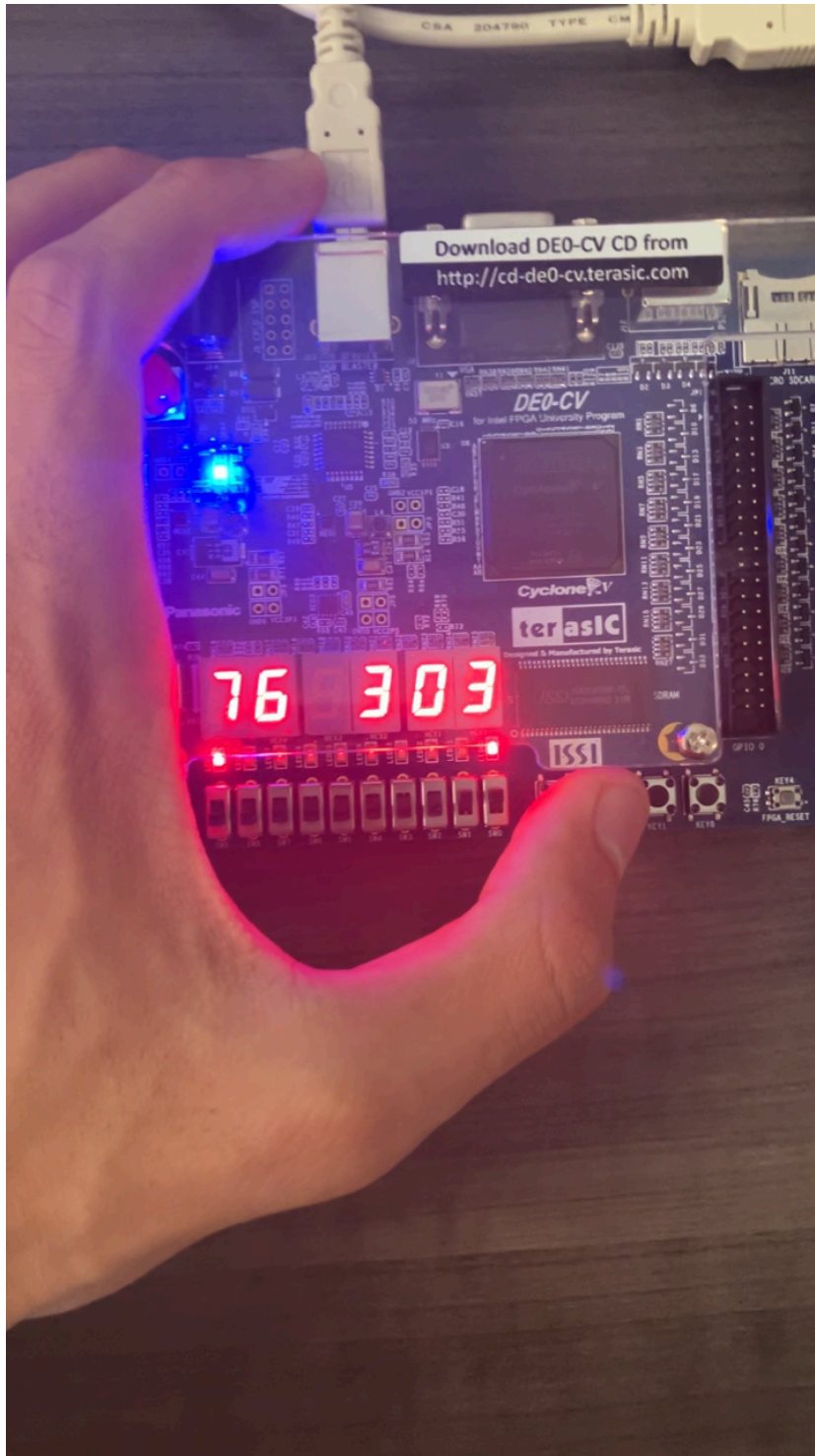


Figure 2.5 : Random Number (0), Player Input (3). Sum = 3

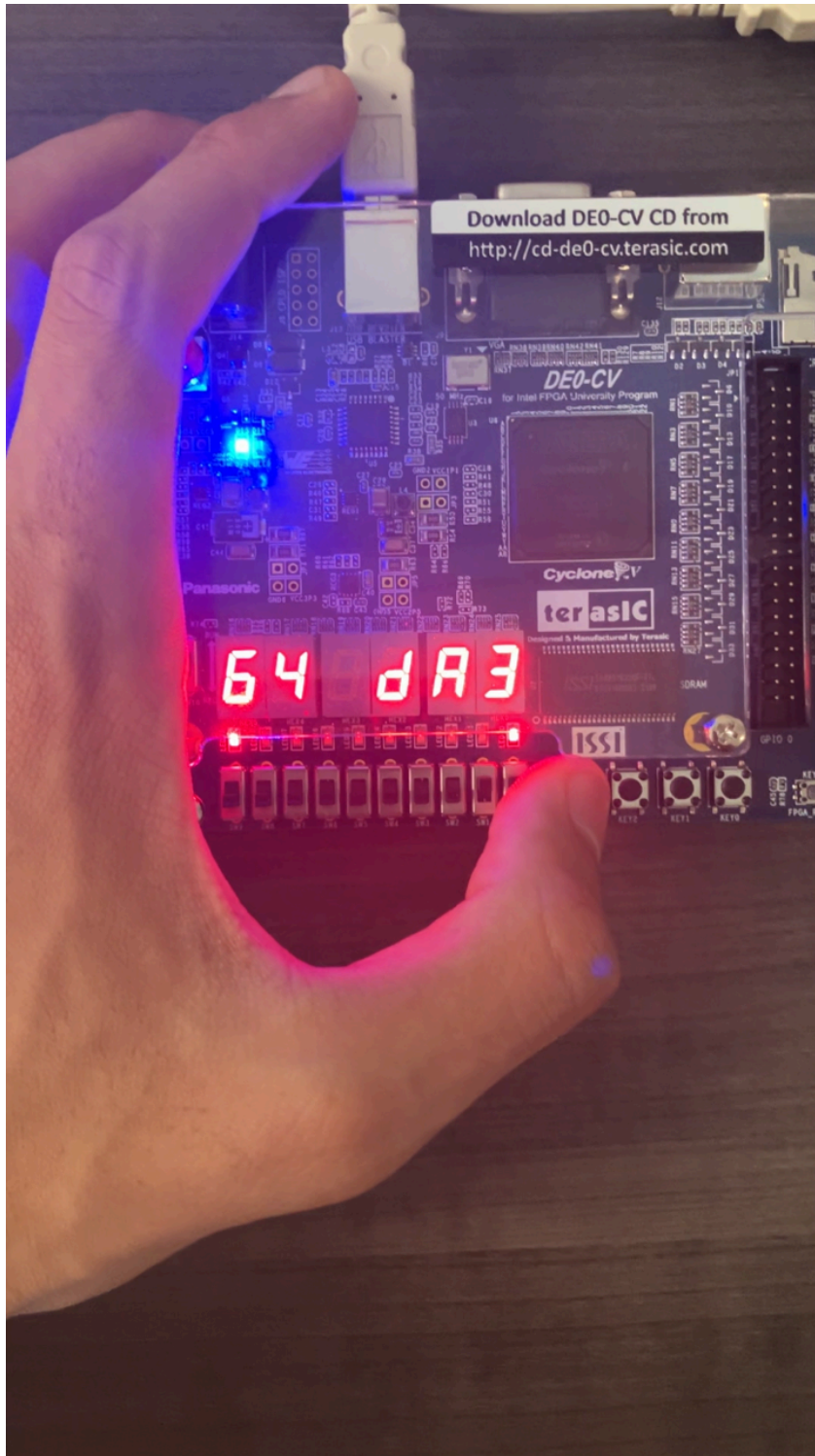


Figure 2.6: 64 seconds remaining Random Number (A - 10 in Hexadecimal), Player Input (3). Sum = 13

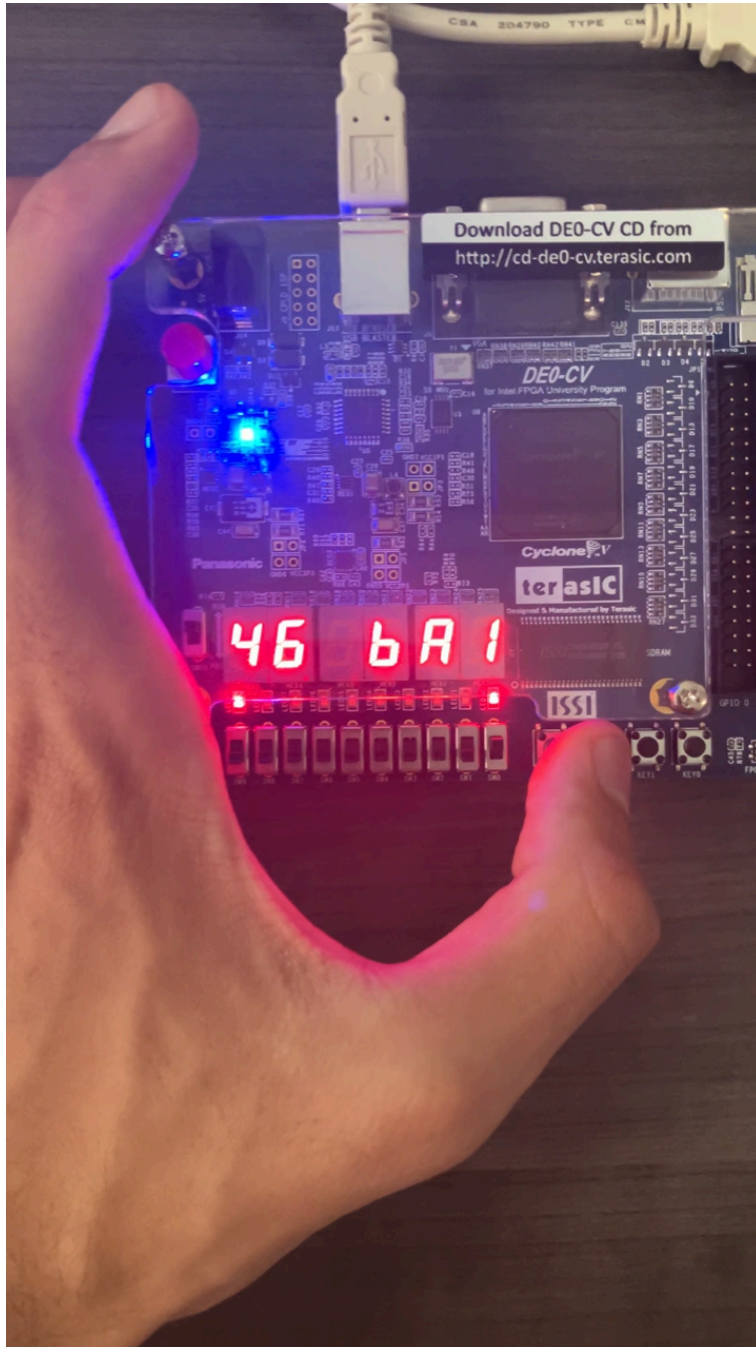


Figure 2.7: 46 seconds remaining Random Number (A - 10 in Hexadecimal), Player Input (1). Sum = 13

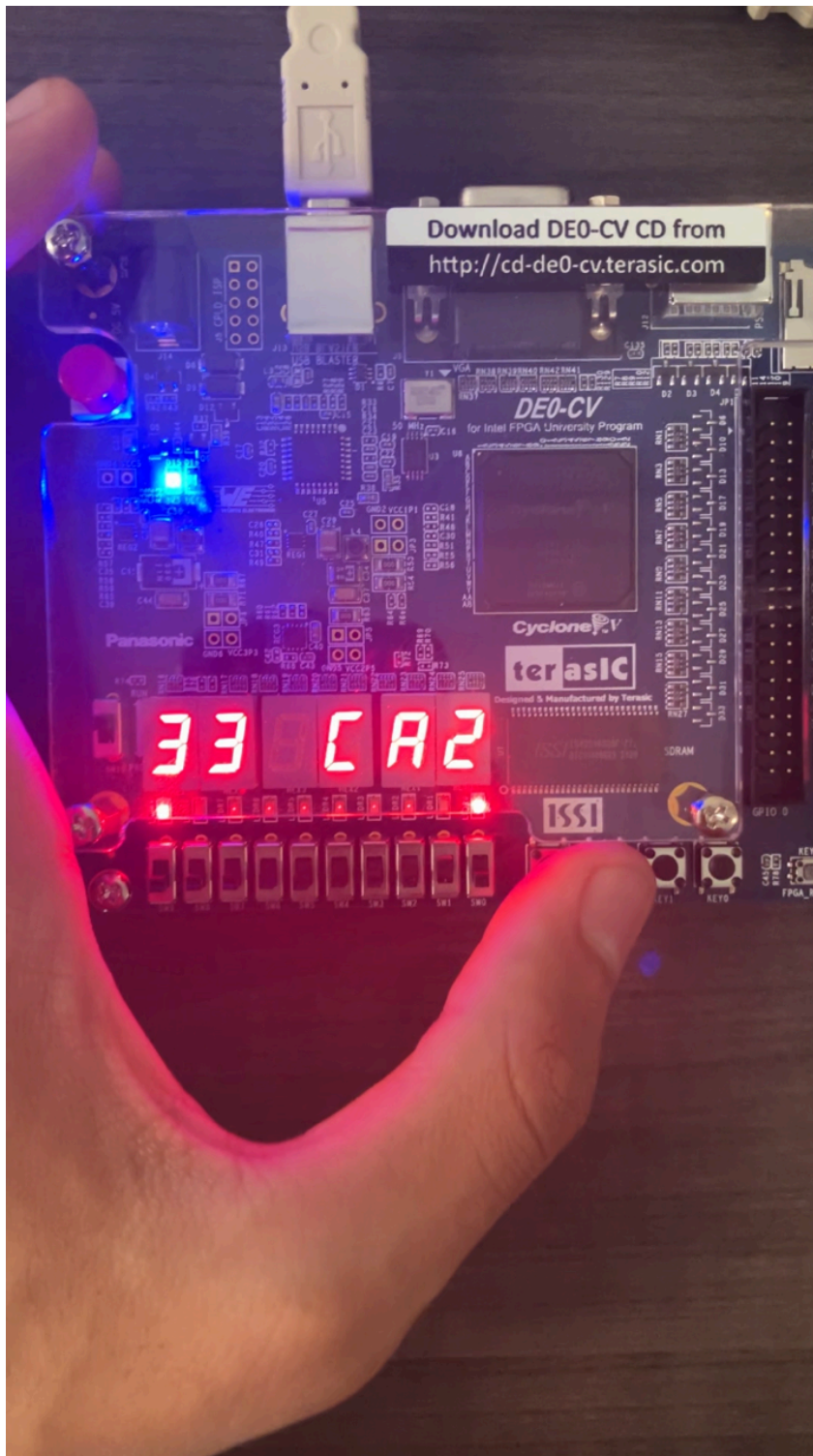


Figure 2.8: 33 seconds remaining Random Number (A - 10 in Hexadecimal), Player Input (C - 12 in Hexadecimal). Sum = 13

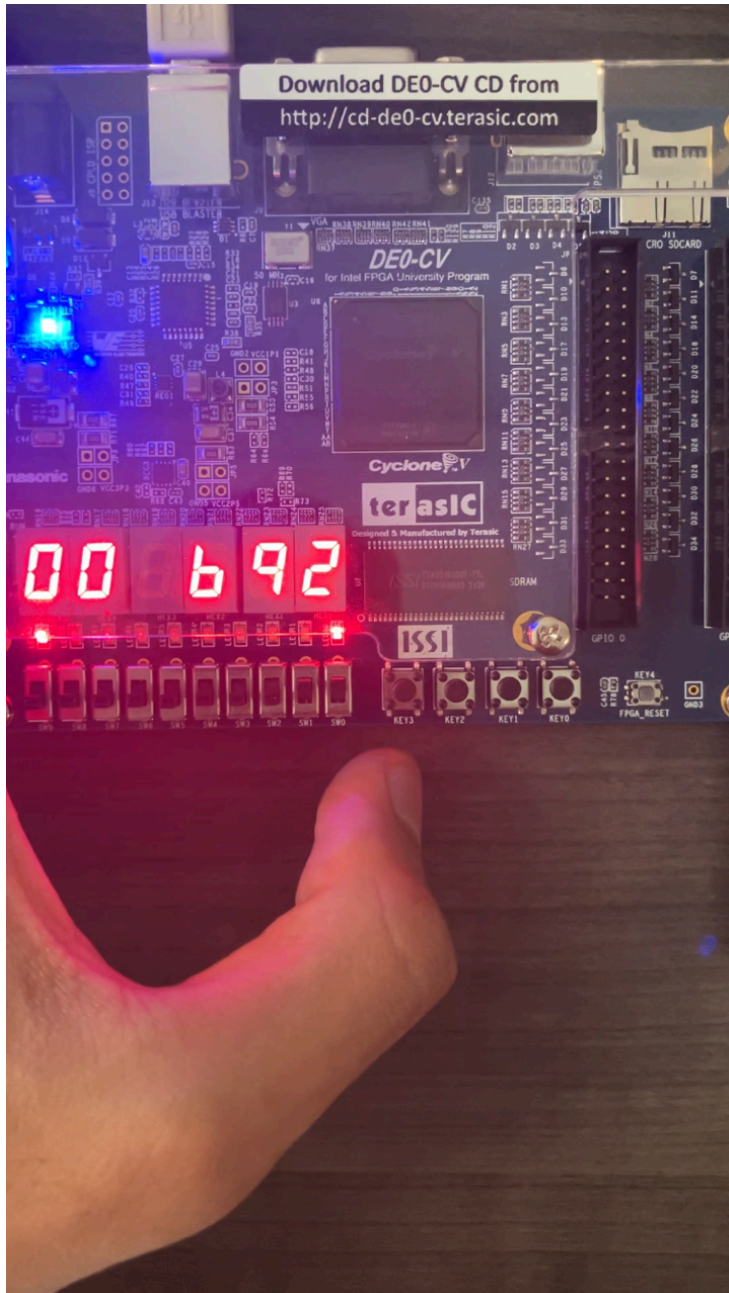


Figure 2.9: 0 seconds remaining Random Number (A - 10 in Hexadecimal), Player Input (1). Sum = B(11 in Hexadecimal)

With zero seconds remaining, the player has two options: quit the game by disassembling the FPGA board, press the reset button, or press the game start button again to play. It's important to note that if the player presses the reset button, his current sum, for instance, shown in Figure 2.9, will not be saved, and all of the displays will be

set to zero. Additionally, the player will have to re-enter the password to play the game again.

However, if the player proceeds to press the game start button again, then his or her current sum, as shown in Figure 2.10 below will be stored and saved with the timer reset to 99 seconds. In this case, the player won't have to re-enter his or her password again.

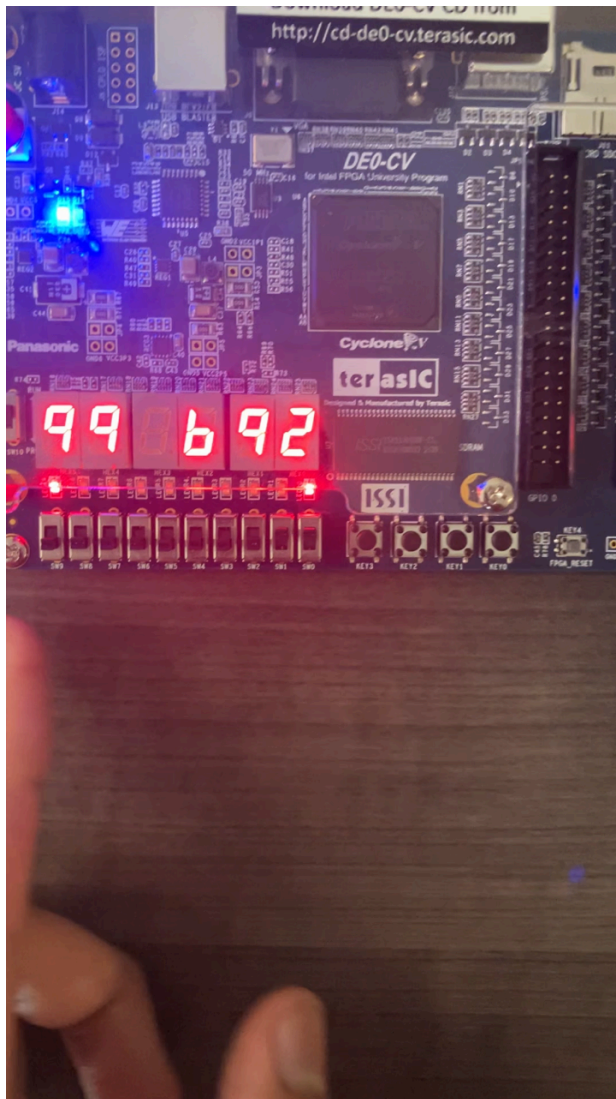


Figure 2.10 : Digit Timer reset to 99 with previous sum inputs saved

For more information about the game played, please watch the video shown in the video link below:

https://drive.google.com/file/d/1teCqafduepYKm2pgmMMAR0a1CtsVKTyh/view?usp=s_haring

***Important to note that the matching LEDs and non-matching LEDs are proof that the random number generated and the player's input can be the same or not. Also note that the voiceover says "Lab 3", but this video applies to Lab 4 since there are minimal changes.**

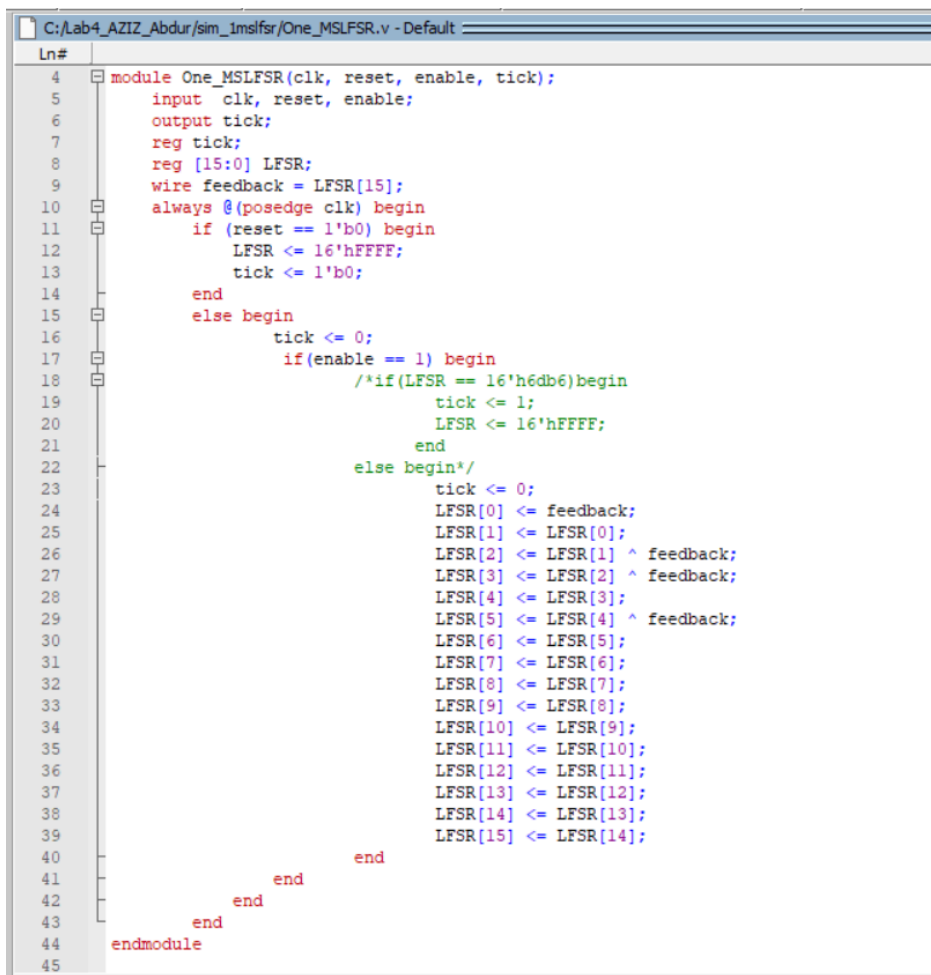
Conclusion:

This lab successfully demonstrated the integration of multiple FPGA hardware modules to implement a secure, interactive, single-player game with real-time control and modular functionality. Expanding upon the foundation established in Lab 2, the Access Controller was restructured into two key modules: the **AuthenticationModule**, which handled secure user verification through ROM-based password storage, and the **GameControllerModule**, which managed the state transitions and timer control for gameplay. This separation of responsibilities not only improved clarity and scalability, but also aligned with best practices in modular digital system design.

Once authenticated, the player participated in a mental math challenge against a simulated bot, with unpredictability introduced by a Linear Feedback Shift Register-based Random Number Generator. The Adder module dynamically computed the sum of the player's input and the bot's random value, while the Four Bit Counter and Digit Timer modules ensured accurate pacing and synchronized feedback through the seven-segment displays. Supporting modules such as the Button Shaper and Load Register maintained signal integrity and input consistency, contributing to a reliable and user-friendly interaction experience.

Overall, Lab 4 reinforced key engineering concepts including FSM-based authentication, modular timer design, synchronous control logic, and clean hardware-software interaction — all critical in the development of robust, real-time digital systems.

Appendix: The Appendix includes non-testbench code first, followed by testbench code second.



```

Ln#
4 module One_MSLFSR(clk, reset, enable, tick);
5   input clk, reset, enable;
6   output tick;
7   reg tick;
8   reg [15:0] LFSR;
9   wire feedback = LFSR[15];
10  always @(posedge clk) begin
11      if (reset == 1'b0) begin
12          LFSR <= 16'hFFFF;
13          tick <= 1'b0;
14      end
15      else begin
16          tick <= 0;
17          if(enable == 1) begin
18              /*if(LFSR == 16'h6db6)begin
19                  tick <= 1;
20                  LFSR <= 16'hFFFF;
21              end
22              else begin*/
23                  tick <= 0;
24                  LFSR[0] <= feedback;
25                  LFSR[1] <= LFSR[0];
26                  LFSR[2] <= LFSR[1] ^ feedback;
27                  LFSR[3] <= LFSR[2] ^ feedback;
28                  LFSR[4] <= LFSR[3];
29                  LFSR[5] <= LFSR[4] ^ feedback;
30                  LFSR[6] <= LFSR[5];
31                  LFSR[7] <= LFSR[6];
32                  LFSR[8] <= LFSR[7];
33                  LFSR[9] <= LFSR[8];
34                  LFSR[10] <= LFSR[9];
35                  LFSR[11] <= LFSR[10];
36                  LFSR[12] <= LFSR[11];
37                  LFSR[13] <= LFSR[12];
38                  LFSR[14] <= LFSR[13];
39                  LFSR[15] <= LFSR[14];
40              end
41          end
42      end
43  end
44 endmodule
45

```

Figure 3.1: Linear Shift Register Based One Second Module before terminal value discovered

```

1 //ECE 5440
2 // Abdur Aziz
3 // ID: 2052062
4 //This is Linear Shift Register Based One Second Timer. It's more efficient than the count to 1ms
5 module One_MSLFSR(clk, reset, enable, tick);
6     input clk, reset, enable;
7     output tick;
8     reg tick;
9     reg [15:0] LFSR;
10    wire feedback = LFSR[15];
11    always @(posedge clk) begin
12        if (reset == 1'b0) begin
13            LFSR <= 16'hFFFF;
14            tick <= 1'b0;
15        end
16        else begin
17            tick <= 0;
18            if(enable == 1) begin
19                if(LFSR == 16'h6db6)begin
20                    tick <= 1;
21                    LFSR <= 16'hFFFF;
22                end
23                else begin
24                    tick <= 0;
25                    LFSR[0] <= feedback;
26                    LFSR[1] <= LFSR[0];
27                    LFSR[2] <= LFSR[1] ^ feedback;
28                    LFSR[3] <= LFSR[2] ^ feedback;
29                    LFSR[4] <= LFSR[3];
30                    LFSR[5] <= LFSR[4] ^ feedback;
31                    LFSR[6] <= LFSR[5];
32                    LFSR[7] <= LFSR[6];
33                    LFSR[8] <= LFSR[7];
34                    LFSR[9] <= LFSR[8];
35                    LFSR[10] <= LFSR[9];
36                    LFSR[11] <= LFSR[10];
37                    LFSR[12] <= LFSR[11];
38                    LFSR[13] <= LFSR[12];
39                    LFSR[14] <= LFSR[13];
40                    LFSR[15] <= LFSR[14];
41                end
42            end
43        end
44    end
45 endmodule

```

Figure 3.2: Linear Shift Register Based One Second Module after terminal value discovered

```

1 //ECE 5440
2 //Abdur Aziz
3 // timer_10ms module
4 // This module counts to 10 millisecond for the timer
5 module timer_10ms (
6     input wire clk,
7     input wire rst,
8     output reg tick
9 );
10
11     reg [3:0] count;
12     always @(posedge clk) begin
13         if (rst == 1'b0) begin
14             count <= 0;
15             tick <= 0;
16         end
17         else begin
18             if (count == 4'b1001) begin
19                 count <= 0;
20                 tick <= 1;
21             end
22             else begin
23                 count <= count + 1;
24                 tick <= 0;
25             end
26         end
27     end
28 endmodule
29

```

Figure 3.3: Counting to 10 millisecond module

```

1 //ECE 5440
2 //Abdur Aziz
3 // CountTo100 module
4 // This module counts to 100 ms for the timer
5 module CountTo100 (
6     input wire clk,
7     input wire rst,
8     output reg tick
9 );
10
11     reg [6:0] count;
12
13     always @(posedge clk) begin
14         if (rst == 1'b0) begin
15             count <= 0;
16             tick <= 0;
17         end
18         else begin
19             if (count == 7'd99) begin
20                 count <= 0;
21                 tick <= 1;
22             end
23             else begin
24                 count <= count + 1;
25                 tick <= 0;
26             end
27         end
28     end
29 endmodule
30

```

Figure 3.4: Counting to 100 millisecond module

```

1 //ECE 5440
2 //Abdur Aziz
3 // One Second Timer module
4 // This module instantiates the 1 ms LFSR and CountTo100 modules so that 1 second of each digit can be outputted
5 // to the digital display. It replaces the 1 ms timer with the 1 ms LFSR
6 module OneSecTimer (
7     input wire clk,
8     input wire reset,
9     input wire enable,
10    output wire timeout
11);
12
13    wire timeoutOneMS;
14    wire timeoutHundredMS;
15
16    One_MSLFSR one_ms1fsr (
17        .clk(clk),
18        .reset(reset),
19        .enable(enable),
20        .tick(timeoutOneMS)
21    );
22
23    CountTo100 timer100MS (
24        .clk(timeoutOneMS),
25        .rst(reset),
26        .tick(timeoutHundredMS)
27    );
28
29    timer_10ms T_10MS (
30        .clk(timeoutHundredMS),
31        .rst(reset),
32        .tick(timeout)
33    );
34
35 endmodule
36

```

Figure 3.5: Revised One Second Timer module

```

1 //ECE 5440
2 //Abdur Aziz
3 //Four Bit Counter Module
4 //This module is the Four Bit Counter. It counts down the digital timer when the user is playing the game
5 module FourBitCounter (
6     input wire clk,
7     input wire rst,
8     input wire Button,
9     output reg [3:0] Count
10);
11
12    always @(posedge clk) begin
13        if (rst == 1'b0)
14            Count <= 4'b0000;
15        else if (Button == 1'b1)
16            Count <= Count + 1;
17    end
18 endmodule
19

```

Figure 3.6: Four Bit Counter module


```

1 //ECE 5440
2 //Abdur Aziz
3 //Random Number Generator Module
4 //This module is the Random Number Generator Module. It generates a random number from 0 - 9 (hexadecimal) and A - F(hexadecimal)
5 module RandomNumberGenerator (
6     input wire clk,
7     input wire rst,
8     input wire button,
9     output reg [3:0] random
10 );
11     wire [3:0] countValue;
12
13     FourBitCounter counter_inst (
14         .clk(clk),
15         .rst(rst),
16         .Button(button),
17         .Count(countValue)
18     );
19
20
21     always @(posedge button) begin
22         if (rst==1'b0)
23             random <= 4'b0000;
24         else
25             random <= countValue;
26     end
27 endmodule
28

```

Figure 3.7: Random Number Generator module

```

1 //ECE 5440
2 //Abdur Aziz
3 //Digit Timer Module
4 //This module enables the timer to decrement by counts of 1 second. When going from the ones 0 to ones 9, we can borrow digits
5 module DigitTimer(
6     input wire clk,
7     input wire rst,
8     input wire reconfig,
9     input wire enable,
10    input wire DownBorrow,
11    input wire NoUpBorrow,
12    output reg UpBorrow,
13    output reg NoDownBorrow,
14    output reg BorrowDown_prev,
15    output reg [3:0] digit
16 );
17     always @(posedge clk) begin
18         if (rst==1'b0) begin
19             // Reset state
20             digit <= 4'b0000;
21             UpBorrow <= 1'b0;
22             NoDownBorrow <= 1'b1;
23             BorrowDown_prev <= 1'b0;
24         end else begin
25             BorrowDown_prev <= DownBorrow;
26             if (reconfig) begin
27                 digit <= 4'd9;
28                 NoDownBorrow <= 1'b1;
29             end
30             else if (enable==1'b1) begin
31                 if (DownBorrow && !BorrowDown_prev) begin
32                     if (digit == 4'd0) begin
33                         digit <= 4'd9;
34                         UpBorrow <= 1'b1;
35                         NoDownBorrow <= 1'b1;
36                     end else begin
37                         digit <= digit - 1'b1;
38                         UpBorrow <= 1'b0;
39                         if ((digit - 1'b1) == 4'd0) begin
40                             NoDownBorrow <= 1'b0;
41                         end else begin
42                             NoDownBorrow <= 1'b1;
43                         end
44                     end
45                 end else begin
46                     UpBorrow <= 1'b0;
47                 end
48             end
49         end
50     end
51 endmodule

```

Figure 3.8: ½ of Digt Timer module

```

6   input wire clk,
7   input wire rst,
8   input wire reconfig,
9   input wire enable,
10  input wire DownBorrow,
11  input wire NoUpBorrow,
12  output reg UpBorrow,
13  output reg NoDownBorrow,
14  output reg BorrowDown_prev,
15  output reg [3:0] digit
16 );
17 always @(posedge clk) begin
18     if (rst==1'b0) begin
19         // Reset state
20         digit <= 4'b0000;
21         UpBorrow <= 1'b0;
22         NoDownBorrow <= 1'b1;
23         BorrowDown_prev <= 1'b0;
24     end else begin
25         BorrowDown_prev <= DownBorrow;
26         if (reconfig) begin
27             digit <= 4'd9;
28             NoDownBorrow <= 1'b1;
29         end
30         else if (enable==1'b1) begin
31             if (DownBorrow && !BorrowDown_prev) begin
32                 if (digit == 4'd0) begin
33                     digit <= 4'd9;
34                     UpBorrow <= 1'b1;
35                     NoDownBorrow <= 1'b1;
36                 end else begin
37                     digit <= digit - 1'b1;
38                     UpBorrow <= 1'b0;
39                     if ((digit - 1'b1) == 4'd0) begin
40                         NoDownBorrow <= 1'b0;
41                     end else begin
42                         NoDownBorrow <= 1'b1;
43                     end
44                 end
45             end else begin
46                 UpBorrow <= 1'b0;
47             end
48         end
49     end
50 end
51 endmodule
52

```

Figure 3.9: Remaining half of Digit Timer module

```

1 // Course Number: ECE 5440
2 // Author: Abdur Aziz, 2062
3 // Module: SevenSegmentDecoder
4 // This is the Seven Segment Decoder as part of Lab 1 - FPGA Mental Binary Game
5
6
7 module SevenSegmentDecoder(decoder_in, decoder_out);
8     output reg [6:0] decoder_out;
9     input [3:0] decoder_in;
10    always@(decoder_in)
11    begin
12
13        case (decoder_in)
14            4'b0000: begin decoder_out = 7'b1000000; end
15            4'b0001: begin decoder_out = 7'b1111001; end
16            4'b0010: begin decoder_out = 7'b0100100; end
17            4'b0011: begin decoder_out = 7'b0110000; end
18
19            4'b0100: begin decoder_out = 7'b0011001; end
20            4'b0101: begin decoder_out = 7'b0010010; end
21            4'b0110: begin decoder_out = 7'b0000010; end
22            4'b0111: begin decoder_out = 7'b1111000; end
23
24            4'b1000: begin decoder_out = 7'b0000000; end
25            4'b1001: begin decoder_out = 7'b0011000; end
26            4'b1010: begin decoder_out = 7'b0000100; end
27            4'b1011: begin decoder_out = 7'b0000011; end
28
29            4'b1100: begin decoder_out = 7'b1000110; end
30            4'b1101: begin decoder_out = 7'b0100001; end
31            4'b1110: begin decoder_out = 7'b0000110; end
32            4'b1111: begin decoder_out = 7'b0000110; end
33            default: begin decoder_out = 7'b1111111; end
34        endcase
35    end
36 endmodule
37

```

Figure 3.10: Seven Segment Decoder Module

```

// Course Number: ECE 5440
// Author: Abdur Aziz, 2062
// Module: 4 bit adder
// This is the 4 bit adder module where the sum of Player 1's inputted binary number and Player 2's binary number is added

module AdderDecoder (A, B, S);
    input [3:0] A, B;
    output reg [3:0] S;
    always@(A,B)
        begin
            S = A + B;
        end
endmodule

```

Figure 4.1: Adder Module

```

1 //Abdur Aziz
2 //ECE 5440
3 //Load Register Module
4 //This module loads in the password (2062) from the player and the number inputs from switches
5 module LoadRegister(D_in, D_out, clk, rst, Load);
6     input [3:0] D_in;
7     output [3:0] D_out;
8     input clk, rst;
9     input Load;
10    reg [3:0] D_out;
11
12    always@(posedge clk)
13    begin
14        if(rst == 1'b0)
15        begin
16            D_out<= 4'b0000;
17        end
18        else
19            //normal operations after reset
20            begin
21                if (Load == 1'b1)
22                begin
23                    D_out <= D_in;
24                end
25            end
26        end
27    end
28 endmodule
29

```

Figure 4.2: Load Register Module in Verilog

```

1 //Abdur Aziz
2 //ECE 5440
3 //Button Shaper Module
4 //This module enables the player to press the buttons to load in numbers and the password(2062)
5 module ButtonShaper(B_in, B_out, clk, rst);
6     input B_in; // Button input signal
7     output B_out; // Button output signal
8     input clk, rst; // Clock and reset signals
9     reg B_out; // Button output register
10    parameter INIT = 2'b00, PULSE = 2'b01, WAIT = 2'b10;
11    reg [2:0] State, StateNext;
12    always @(posedge clk or posedge rst) begin
13        if (rst) begin
14            State <= INIT;
15        end else begin
16            State <= StateNext;
17        end
18    end
19    always @(State, B_in) begin
20        case(State)
21            INIT: begin
22                B_out = 1'b0;
23                if (B_in == 1'b0)
24                    StateNext = PULSE;
25                else
26                    StateNext = INIT;
27            end
28        PULSE: begin
29            B_out = 1'b1;
30            StateNext = WAIT;
31        end
32        WAIT: begin
33            B_out = 1'b0;
34            if (B_in == 1'b0)
35                StateNext = WAIT;
36            else
37                StateNext = INIT;
38        end
39        default: begin
40            StateNext = INIT;
41            B_out = 1'b0;
42        end
43    endcase
44 end
45 endmodule

```

Figure 4.3: Button Shaper Module in Verilog

```

1 //ECE 5440
2 //Abdur Aziz
3 //Game Controller Module
4 //This module is the Game Controller Controller module. It's used to control the states of the game if it's playing or over
5 module GCM (
6     input wire clk,
7     input wire rst, // Active-low reset
8     input wire PasswordEnterButton, // Used to move through gameplay states
9     input wire LoggedIn, // From Authentication
10    input wire Timeout, // From 99s timer
11    output reg Timer_reconfig, // Pulse to reset timer
12    output reg Timer_enable, // Enables 1-second timer
13);
14
15 parameter INACTIVE = 0, //Game is not playing
16 PASSED = 1, //Game control password is correct
17 RECONFIG = 2, // We reconfigure the timer
18 GAMEPLAY = 3, // We play the game
19 GAMEOVER = 4; // The game is over
20 reg [2:0] State;
21
22 always @(posedge clk) begin
23     if (rst == 1'b0) begin
24         State <= INACTIVE;
25         Timer_reconfig <= 0;
26         Timer_enable <= 0;
27     end else begin
28         // Default outputs
29         Timer_reconfig <= 0;
30         Timer_enable <= 0;
31         case (State)
32             INACTIVE: begin
33                 if (LoggedIn)
34                     State <= PASSED;
35             end
36             PASSED: State <= RECONFIG;
37             RECONFIG: begin
38                 Timer_reconfig <= 1;
39                 if (PasswordEnterButton)
40                     State <= GAMEPLAY;
41             end
42             GAMEPLAY: begin
43                 Timer_enable <= 1;
44                 if (Timeout)
45                     State <= GAMEOVER;
46             end
47         endcase
48     end
49 end
50 endmodule

```

Figure 4.4: ½ of the Game Controller Module in Verilog

```

48 GAMEOVER: begin
49     Timer_enable <= 0;
50     if (PasswordEnterButton)
51         State <= RECONFIG;
52     end
53 default:
54     State <= INACTIVE;
55 endcase
56 end
57 end
58
59 endmodule

```

Figure 4.5: Remaining half of the Game Controller Module in Verilog


```

1 //ECE 5440
2 //Abdur Aziz
3 //Authentication Module
4 // This is the remaining module separated from the AccessController module
5 // as per Lab 4 instructions
6 // It loads and stores the password into the ROM
7 module AuthenticationModule (
8     input wire rst,
9     input wire clk,
10    input wire PasswordEnterButton,
11    input wire [3:0] PasswordDigit,
12    output reg LoggedIn,
13    output reg LoggedOut
14);
15
16    reg PassVerified;
17    reg compareflag;
18    reg [1:0] State;
19    reg [1:0] counter;
20    wire [3:0] digitnew;
21    parameter DIGITS = 0,
22    VERIFY = 1;
23
24    // Instantiate ROM
25    ROM_PSWD rom_pswd (
26        .address({3'b000, counter}),
27        .clock(clk),
28        .q(digitnew)
29    );
30
31    always @(posedge clk) begin
32        if (rst==1'b0) begin
33            State <= DIGITS;
34            counter <= 0;
35            PassVerified <= 1;
36            LoggedIn <= 0;
37            LoggedOut <= 1;
38            compareflag <= 0;
39        end else begin
40            case (State)
41            DIGITS: begin
42                if (PasswordEnterButton == 1'b1) begin
43                    compareflag <= 1;
44                end else if (compareflag==1'b1) begin
45                    PassVerified <= PassVerified & (PasswordDigit == digitnew);
46                    counter <= counter + 1;
47                    compareflag <= 0;
48                    if (counter == 2'b11) begin

```

Figure 4.6: 1st half of the Authentication Module

```

48         if (counter == 2'b11) begin
49             State <= VERIFY;
50         end
51     end
52 end
53 VERIFY: begin
54     if (PassVerified == 1'b1) begin
55         LoggedIn <= 1;
56         LoggedOut <= 0;
57     end else begin
58         LoggedIn <= 0;
59         LoggedOut <= 1;
60     end
61     State <= DIGITS;
62     counter <= 0;
63     PassVerified <= 1;
64 end
65 default:
66     State <= DIGITS;
67 endcase
68 end
69 end
70
71 endmodule

```

Figure 4.7: Remaining half of the Authentication Module

```

1 //ECE 5440
2 //Abdur Aziz
3 // Lab 4 Top Module
4 // This Lab declares and instantiates all of the src folders to program on the FPGA Board.
5 module Lab4_AZIZ_Abdur(
6     input clk,
7     input rst,
8
9     input [3:0] PasswordInputSwitches, // Password digit input
10    input [3:0] PlayerInputSwitches, // Player can switch toggles for numbers
11    input RNGButtonPress, // Player pushes button to generate random number
12    input PasswordButtonPress, // Player pushes button to enter password
13    input PlayerButtonPress, // Player presses this button to load number
14
15
16    // Timer Display Outputs
17    output [6:0] seg_ten, // Tens digit
18    output [6:0] seg_one, // Ones digit
19
20    output [6:0] seg_player,
21    output [6:0] seg_rng,
22    output [6:0] seg_sum,
23
24    // Status LEDs
25    output LED1st, // Logged in
26    output LED2nd, // Logged out - Added this for debugging purposes, great to check if program is working or not
27    output Matches,
28    output NotMatches
29 );
30
31 wire Timer_reconfig, Timer_enable;
32 wire LoggedIn, LoggedOut;
33 wire timeout_1sec;
34
35 // Internal Wires
36 wire PasswordShapedButton; //shaped password entry button
37 wire PlayerShapedButton; // shaped player load button
38 wire [3:0] SwitchesOutput; //this is added to random number in 4 bit adder
39 wire [3:0] random; //Shows Random Number
40 wire [3:0] result; //Shows LEDs
41 wire UpBorrow_Ones, NoDownBorrow_Ones;
42 wire NoDownBorrow_Tens;
43 wire [3:0] tens;
44 wire [3:0] ones;
45
46 // Assign LEDs to show them when they turn on or off
47 assign LED1st = LoggedIn;
48 assign LED2nd = LoggedOut;

```

Figure 4.8:¼ of the Lab 4 Top Module in Verilog

```

49
50 assign Matches = (result == 4'b1111);
51 assign NotMatches = ~(result == 4'b1111);
52
53 // Button Shaper modules for Password and Players
54 ButtonShaper PasswordLoad(
55     .B_in>PasswordButtonPress),
56     .B_out>PasswordShapedButton),
57     .clk(clk),
58     .rst(rst)
59 );
60
61 ButtonShaper PlayerLoad(
62     .B_in>PlayerButtonPress),
63     .B_out>PlayerShapedButton),
64     .clk(clk),
65     .rst(rst)
66 );
67
68 AdderDecoder add(
69     .A(SwitchesOutput),
70     .B(RandomNumber),
71     .S(result)
72 );
73
74 SevenSegmentDecoder displayPlayer(
75     .decoder_in>SwitchesOutput),
76     .decoder_out>seg_player)
77 );
78
79 SevenSegmentDecoder displayRNG(
80     .decoder_in>random),
81     .decoder_out>seg_rng)
82 );
83
84 SevenSegmentDecoder displaySum(
85     .decoder_in>result),
86     .decoder_out>seg_sum)
87 );
88
89 // I decided to create two new wires based on the status of the ButtonShapers and the timer enable button
90 wire LoadRandomNumber = RNGButtonPress & Timer_enable;
91
92 wire EnablePlayer = Timer_enable & PlayerShapedButton;
93
94 LoadRegister LoadPlayer(PlayerInputSwitches, SwitchesOutput, clk, rst, EnablePlayer);
95

```

Figure 4.9 ½ of the Lab 4 Top Module in Verilog

```

96
97 RandomNumberGenerator RandNum(
98     .clk(clk),
99     .rst(rst),
100     .button(LoadRandomNumber),
101     .random(random)
102 );
103
104 OneSecTimer onesec(
105     .clk(clk),
106     .reset(rst),
107     .enable(Timer_enable),
108     .timeout(timeout_1sec)
109 );
110 DigitTimer onesDigit(
111     .clk(clk),
112     .rst(rst),
113     .reconfig(Timer_reconfig),
114     .enable(Timer_enable),
115     .DownBorrow(timeout_1sec),
116     .NoUpBorrow(NoDownBorrow_Tens),
117     .UpBorrow(UpBorrow_Ones),
118     .NoDownBorrow(NoDownBorrow_Ones),
119     .digit(ones)
120 );
121
122 DigitTimer tensDigit(
123     .clk(clk),
124     .rst(rst),
125     .reconfig(Timer_reconfig),
126     .enable(Timer_enable),
127     .DownBorrow(UpBorrow_Ones),
128     .NoUpBorrow(1'b1),
129     .UpBorrow(),
130     .NoDownBorrow(NoDownBorrow_Tens),
131     .digit(tens)
132 );
133
134
135 SevenSegmentDecoder showTens(
136     .decoder_in(tens),
137     .decoder_out(seg_ten)
138 );
139
140 SevenSegmentDecoder showOnes(
141     .decoder_in(ones),
142     .decoder_out(seg_one)
143 );
144

```

Figure 4.10 ¾ of the Lab 4 Top Module in Verilog


```

145 AuthenticationModule authenticationmodule (
146     .clk(clk),
147     .rst(rst),
148     .PasswordEnterButton>PasswordShapedButton),
149     .PasswordDigit>PasswordInputSwitches),
150     .LoggedIn(Login),
151     .LoggedOut(LoggedOut)
152 );
153
154 GCM gcm (
155     .clk(clk),
156     .rst(rst),
157     .PasswordEnterButton>PasswordShapedButton),
158     .LoggedIn(Login),
159     .Timeout(~NoDownBorrow_Tens && ~NoDownBorrow_Ones),
160     .Timer_reconfig(Timer_reconfig),
161     .Timer_enable(Timer_enable)
162 );
163
164 endmodule
165
166

```

Figure 5.1: Remaining Lab 4 Top Module in Verilog

This concludes the non-testbench code. Below commences the testbench code for the relevant modules.

```

module testbench_OneSecTimer;
    reg clk, reset, enable;
    wire timeout;

    OneSecTimer oneSec(.clk(clk), .reset(reset), .enable(enable), .timeout(timeout));

    always #10 clk = ~clk;

    initial begin
        //initialize signals
        clk = 0;
        reset = 1;
        enable = 0;

        #50
        reset = 0;

        #30
        enable = 1;

        #5000

        $stop;
    end
endmodule

```

Figure 5.2: One Second Timer Testbench

```
C:/Lab4_AZIZ_Abdur/sim_1mslfsr/One_MSLFSR.v (/testbench_One_MSLFSR/DUT_One_MSLFSR)
C:/Lab4_AZIZ_Abdur/sim_1mslfsr/testbench_One_MSLFSR.v (/testbench_One_MSLFSR)
File Edit View Tools Bookmarks Window Help
C:/Lab4_AZIZ_Abdur/sim_1mslfsr/testbench_One_MSLFSR.v (/testbench_One_MSLFSR) - Default

Ln#
1 `timescale 1 ns/100 ps
2
3 module testbench_One_MSLFSR();
4     reg Count;
5     reg clk, rst;
6
7     wire Timeout_Oldpulse, Timeout_Newpulse;
8
9     initial begin
10        clk = 1'b0;
11        forever begin
12            #10 clk = ~clk;
13        end
14    end
15    oneMSTimer DUT_oneMSTimer(
16        .clk(clk),
17        .reset(rst),
18        .enable(Count),
19        .tick(Timeout_Oldpulse)
20    );
21    One_MSLFSR DUT_One_MSLFSR(
22        .clk(clk),
23        .reset(rst),
24        .enable(Count),
25        .tick(Timeout_Newpulse)
26    );
27    initial begin
28        rst = 1'b1;
29        Count = 1'b0;
30
31        @(posedge clk);
32        @(posedge clk);
33        #5 rst = 1'b0; // Reset values make all values go to default
34        @(posedge clk);
35        @(posedge clk);
36        @(posedge clk);
37        #5 rst = 1'b1;
38        @(posedge clk);
39        @(posedge clk);
40        #5 Count = 1'b1;
41    end
42 endmodule
43
```

Figure 5.3: LFSR testbench Module

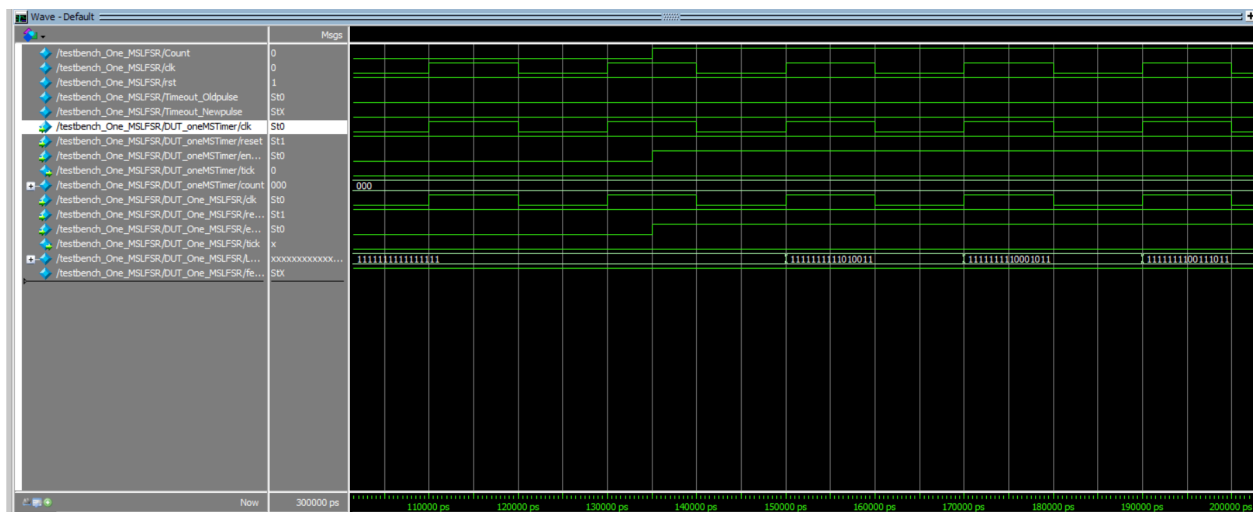


Figure 5.5 : 2nd half of simulation of LFSR 1ms testbench without terminal value



Figure 5.6 : 1st half of simulation of LFSR 1ms testbench with terminal value

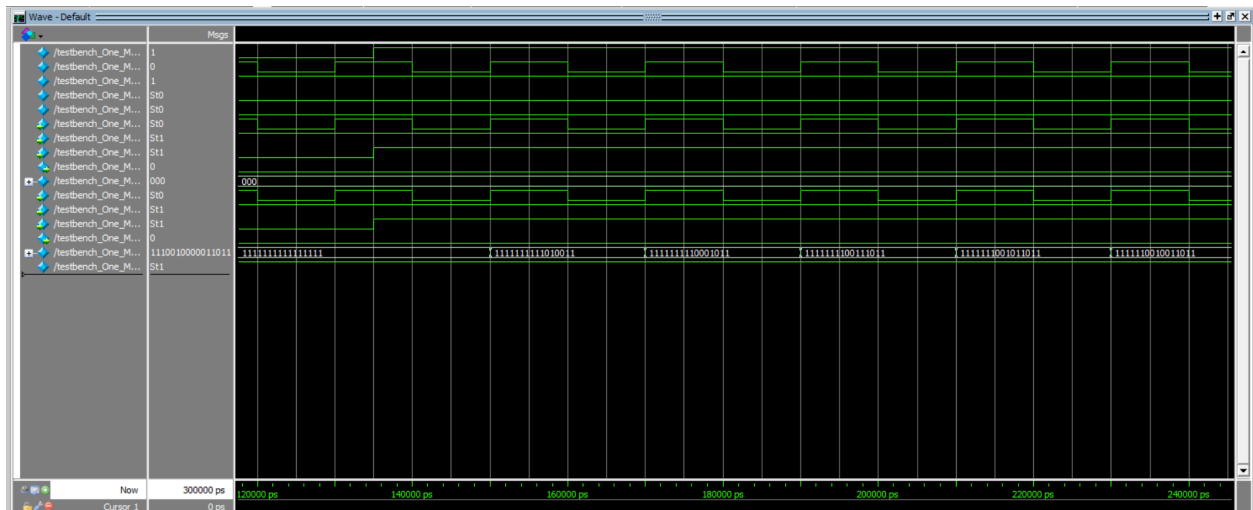


Figure 5.7 : 2nd half of simulation of LFSR 1ms testbench with terminal value